

VIETNAM NATIONAL UNIVERSITY HCMC
UNIVERSITY OF INFORMATION TECHNOLOGY



COURSE
SE100 - Object-Oriented Analysis and Design

Topic
Online shop management software

Teacher:
Lê Thanh Trọng

Group:
2352 xxxx Placeholder1
2352 xxxx Placeholder2
2352 xxxx Placeholder3
2352 xxxx Placeholder4
2352 xxxx Placeholder5

Acknowledgments

We would like to express our sincere gratitude to our supervisor, **Lê Thanh Trọng**, for his guidance and support throughout this project.

We would also like to thank the University of Information Technology, VNU-HCM, for providing the necessary resources and environment for us to complete this course.

Finally, we are grateful for the collaboration and hard work of all team members in making this project a reality.

Abstract

“The Everything Shop” is a functional e-commerce web application developed as a semester project for the Object-Oriented Analysis and Design course.

The primary goal of this project is to apply theoretical knowledge of software engineering to build a complete Model-View-Controller (MVC) system. The application allows users to browse products, manage a shopping cart, and place orders. It allows administrators to manage products and view orders.

The system is built using the PERN stack (PostgreSQL, Express, React, Node.js) with TypeScript, ensuring type safety and code maintainability. Docker is used to containerize the application for consistent development environments.

This report documents the analysis, design, and implementation of the system, highlighting the practical application of design patterns and web development best practices.

Problem Statement

In the rapidly evolving field of software development, understanding the core principles of web application architecture is crucial. While many tools and frameworks exist to automate e-commerce site creation, building one from scratch provides invaluable insights into:

1. **State Management:** Handling user sessions, shopping carts, and authentication across a stateless protocol like HTTP.
2. **Data Modeling:** Designing a relational database schema that accurately represents products, users, orders, and their relationships.
3. **MVC Architecture:** Separating concerns between the database (Model), the user interface (View), and the business logic (Controller).

Therefore, this project aims to address the educational problem of bridging the gap between theoretical software design and practical web application development by implementing a fully detailed e-commerce system.

System Objectives

The system objectives are divided into functional objectives, representing the features available to users, and educational objectives, representing the learning outcomes of the project.

Functional Objectives:

1. **Product Management:** Allow administrators to create, update, and delete products with images and details.
2. **Shopping Experience:** Enable customers to browse products, view details, and manage a shopping cart.
3. **Order Processing:** Provide a checkout mechanism for users to place orders and for administrators to view them.
4. **User Authentication:** Secure the application with registration and login functionality using JWT.

Educational Objectives:

1. **Full-Stack Integration:** Successfully connect a React frontend with an Express backend and a PostgreSQL database.
2. **MVC Implementation:** Demonstrate a clear separation of concerns using the Model-View-Controller pattern.
3. **Database Design:** Implement a relational schema with proper foreign key constraints and relationships.

Proposed Solution

To achieve the stated objectives, we propose a web-based “Online Shop Management Software” built on a **Monolithic MVC Architecture**. This approach centralizes business logic and data access, offering the following advantages for this academic context:

- **Simplicity:** Easier to develop, debug, and deploy as a single unit or closely coupled services.
- **Data Integrity:** Unified database simplifies transaction management and data consistency.
- **Learning Curve:** Ideal for understanding the fundamental request-response cycle and full-stack flow.

The solution comprises three distinct layers:

1. **Presentation Layer (View):** A React Single Page Application (SPA) that provides a dynamic and responsive user interface.
2. **Logic Layer (Controller):** A Node.js/Express server that handles API requests, processes business logic, and interacts with the database.
3. **Data Layer (Model):** A PostgreSQL database accessed via Prisma ORM for structured data storage.

Technologies Used

The project utilizes the **PERN stack**, a popular choice for modern web development, aimed at consistency (JavaScript/TypeScript everywhere) and performance.

Frontend:

- **React**: For building the component-based user interface.
- **Redux**: For global state management (user session, cart).
- **Vanilla CSS**: For styling the application.

Backend:

- **Node.js & Express**: For the server-side runtime and RESTful API framework.
- **TypeScript**: To add static typing to JavaScript, reducing runtime errors.
- **Prisma**: As an Object-Relational Mapper (ORM) to simplify database interactions.
- **JWT (JSON Web Tokens)**: For secure, stateless authentication.

Database & Infrastructure:

- **PostgreSQL**: A powerful, open-source relational database.
- **Docker**: To containerize the backend, frontend, and database, ensuring the application runs consistently across different development machines.

Achieved Results

The “Everything Shop” project has been successfully implemented with the following key results:

1. **Functional Web Application:** A fully deployable e-commerce site where users can register, login, browse products, and simulate purchases.
2. **Complete MVC Flow:** The codebase demonstrates a clear flow of data from the database (Postgres) through the backend controllers (Express) to the frontend UI (React).
3. **Database Integration:** Successful integration of Prisma ORM with PostgreSQL.
4. **Containerized Environment:** A working `docker-compose` setup that orchestrates the app, database, and auxiliary services like Mailhog (for email testing) and MinIO (for file storage simulation).

The project fulfills the requirements of the course by demonstrating a working knowledge of full-stack software development principles.

Table of Contents

1. INTRODUCTION	14
1.1. Necessity of the Topic	14
1.2. System Objectives	14
1.3. Research Scope and Limitations	15
1.4. User Scope and Audience	15
1.5. Methodology	16
1.6. Report Structure	16
2. THEORETICAL BASIS AND TECHNOLOGY	18
2.1. Theoretical Basis	18
2.1.1. Model-View-Controller (MVC) Pattern	18
2.1.2. RESTful API	18
2.1.3. Single Page Application (SPA)	18
2.2. Overview of Related Technologies	18
2.2.1. Frontend	18
2.2.2. Backend	18
2.2.3. Database	19
2.2.4. DevOps & Infrastructure	19
2.3. Rationale for Technology Selection	19
2.3.1. Why PERN Stack (Postgres, Express, React, Node)?	19
2.3.2. Why TypeScript?	19
2.3.3. Why Containerization?	19
3. SYSTEM REQUIREMENTS ANALYSIS	20
3.1. Problem Description and Business Logic	20
3.1.1. Problem Description	20
3.1.2. Business Logic Analysis	20
3.2. Stakeholders	20
3.3. Functional Requirements	21
3.3.1. User Management (EP-01)	21
3.3.2. Product Management (EP-02)	21
3.3.3. Shopping & Ordering (EP-03, EP-04)	21
3.3.4. Promotions & Returns (EP-06, EP-07)	21
3.3.5. Reporting (EP-08)	21
3.4. Non-Functional Requirements	21
3.4.1. Performance Requirements	21
3.4.2. internal/Reliability & Availability	21
3.4.3. Security Requirements	22
3.4.4. Usability	22
3.5. Use Case Diagram	22
3.5.1. Use Case Diagrams	22
3.6. Use Case Specifications	26
3.6.1. User Management	27
3.6.2. Product Management	29
3.6.3. Shopping & Browsing	31
3.6.4. Ordering Process	32
3.6.5. Order Management	34
3.6.6. Promotions & Reporting	35
4. SYSTEM DESIGN	37
4.1. Overall Architecture Design	37

4.1.1.	Overall Architecture Design	37
4.1.1.1.	Presentation Layer (Frontend)	37
4.1.1.2.	Application Layer (Backend)	37
4.1.1.3.	Data Layer (Database)	37
4.2.	Functional Design	38
4.2.1.	Functional Design and Module Details	38
4.2.1.1.	Authentication Module	38
4.2.1.2.	Product Management Module	42
4.2.1.3.	Order Management Module	45
4.2.1.4.	Payment Module	50
4.2.1.5.	Promotion Module	51
4.2.1.6.	Reporting & Analytics Module	52
4.3.	Data Design	53
4.3.1.	Data Design	53
4.3.1.1.	Key Entities	53
5.	IMPLEMENTATION AND DEPLOYMENT	55
5.1.	Development Environment	55
5.2.	Software Configuration	55
5.3.	Development Tools	55
5.4.	Source Code Structure	55
5.5.	Project Tree	55
5.6.	Backend Architecture	56
5.7.	Frontend Architecture	56
5.8.	Implementation of Main Functions	56
5.8.1.	Authentication Module	56
5.8.1.1.	Login Controller	56
5.8.2.	Product Management	57
5.8.2.1.	Get All Products	57
5.8.3.	Order Module	58
5.8.3.1.	Create Direct Order	58
5.8.4.	Cart Management	58
5.8.4.1.	Checkout Logic	58
5.8.5.	Security Middleware	59
5.8.5.1.	Auth Guard	59
5.9.	System Deployment	59
5.9.1.	Deployment Architecture	59
5.9.1.1.	Core Services	59
5.9.2.	Docker Configuration	60
5.9.2.1.	Production Compose Stack	60
5.9.2.2.	Immutable Artifacts (Dockerfile)	60
5.9.3.	Environment Configuration	60
5.9.4.	Deployment Diagram (Textual Representation)	61
6.	TESTING AND EVALUATION	62
6.1.	Testing Strategy	62
6.1.1.	Testing Levels	62
6.1.1.1.	Unit Testing	62
6.1.1.2.	Integration Testing	62
6.1.1.3.	System Testing	62
6.1.2.	Testing Environment	62
6.2.	Test Case Construction and Execution	62
6.2.1.	Authentication Module Tests	62

6.2.2. Product Module Tests	63
6.2.3. Ordering & Checkout Tests	63
6.2.4. Security & Permissions Tests	64
7. CONCLUSION AND FUTURE DEVELOPMENT	65
7.1. Conclusion	65
7.2. Limitations of the Project	65
7.3. Future Development	65
8. REFERENCES	67

List of Figures

Figure 1	Epic 01: Account & Session	22
Figure 2	Epic 02: Product Management	23
Figure 3	Epic 03: Browsing & Searching	23
Figure 4	Epic 04: Order Management	24
Figure 5	Epic 05: Payment	24
Figure 6	Epic 06: Promotions	25
Figure 7	Epic 07: Returns	25
Figure 8	Epic 08: Reporting	26
Figure 9	System Architecture Diagram	38
Figure 10	Registration Sequence	39
Figure 11	Login Sequence	39
Figure 12	Verify Email Sequence	40
Figure 13	Forgot Password Sequence	40
Figure 14	Reset Password Sequence	41
Figure 15	Token Refresh Sequence	41
Figure 16	Logout Sequence	42
Figure 17	Create Product Sequence	42
Figure 18	Search & Filter Sequence	43
Figure 19	Update Stock Sequence	43
Figure 20	Get Product Detail Sequence	44
Figure 21	Delete Product Sequence	44
Figure 22	Get Seller Products Sequence	45
Figure 23	Add to Cart Sequence	46
Figure 24	View Cart Sequence	46
Figure 25	Update Cart Item Sequence	47
Figure 26	Remove from Cart Sequence	47
Figure 27	Order Checkout Sequence	48
Figure 28	Order History Sequence	48
Figure 29	Order Detail Sequence	49
Figure 30	Update Order Status Sequence	49
Figure 31	Cancel Order Sequence	50
Figure 32	Payment Initiation Sequence	50
Figure 33	Create Promotion Sequence	51

Figure 34 Create Coupon Sequence	52
Figure 35 List Promotions Sequence	52
Figure 36 Sales Report Sequence	53
Figure 37 Entity Relationship Diagram	54

1. INTRODUCTION

1.1. Necessity of the Topic

The development of an “Online Shop Management Software” was assigned as the core project for the Object-Oriented Analysis and Design course. This topic serves as a fundamental exercise in software engineering, requiring students to translate theoretical concepts into a tangible, functioning system.

While the market is flooded with commercial e-commerce solutions, building one from the ground up offers a unique educational vantage point. It compels us to grapple with essential challenges such as:

- **Data Integrity and Modeling:** Designing a relational database that can robustly handle interconnected entities like Users, Products, Orders, and Reviews without data anomalies.
- **Full-Stack Architecture:** Bridging the gap between a dynamic frontend (React) and a stateless RESTful API (Express), ensuring seamless data flow and user experience.
- **State Management:** Implementing persistent user sessions and shopping cart functionality in a stateless HTTP environment.
- **Security Fundamentals:** Practical application of authentication (JWT) and authorization (RBAC) to protect user data and administrative functions.

Therefore, the necessity of this project lies not in creating a novel product for the market, but in mastering the fundamental patterns and technologies that underpin modern web application development.

1.2. System Objectives

The primary objective is to successfully deliver a web-based “Everything Shop” application that meets the course requirements. This involves dual goals: functioning software and educational mastery.

General Objectives:

- **Application Development:** Build a robust, responsive web application that simulates a typical B2C e-commerce workflow.
- **Architectural Design:** Apply the Model-View-Controller (MVC) pattern to separate concerns, ensuring code maintainability and scalability.
- **Infrastructure:** Demonstrate competence in modern deployment technologies by containerizing the application with Docker.

Specific Objectives:

- **User Account Management:**
 - Guest Registration and Secure Login (JWT).
 - Profile Management (Update details).
 - Role-based Access Control (Customer vs Seller vs Admin).
- **Product Discovery:**
 - Browse Products by Category hierarchy.
 - Search functionality for finding items.
 - View detailed product specifications.
- **Shopping and Checkout:**
 - Persistent shopping cart management.
 - Order creation and simulation of payment processing.
 - Coupon code application logic.

- **Management:**
 - Admin dashboard for user and system oversight.
 - Seller tools for managing inventory and order statuses., Delete) operations for Products.
 - Ability to view customer orders and update their status (e.g., Pending -> Shipped).

1.3. Research Scope and Limitations

To ensure the project is achievable within the semester's constraints, we have clearly defined the boundaries of the system.

Research Scope (What is included):

- **Business Model:** The system operates on a Business-to-Consumer (B2C) model. A single entity (the "Shop") sells products directly to multiple end-users ("Customers"). Multi-vendor marketplaces (C2C) are out of scope.
- **Core Modules:**
 - **Authentication Module:** Registration, Login, Logout, Password Hashing.
 - **Product Module:** Categories, Product Listings, Inventory counts.
 - **Order Module:** Cart management, Order creation, Order History.
 - **Admin Module:** Administrative interface for content management.
- **Technology Stack:** strictly adhered to the PERN stack (Postgres, Express, React, Node) with Docker for environment orchestration.

Limitations (What is excluded):

- **Payment Processing:** The system simulates the checkout process up to the point of "Placing Order". Integration with real-world payment gateways (Stripe, PayPal, VNPay) is simulated or omitted to avoid financial complexity.
- **Advanced Search:** Product search is implemented with basic string matching, lacking features like fuzzy search or relevance ranking.
- **Shipping and Logistics:** Shipping logic is simplified. Real-time shipping rate calculation and tracking integration are not included.
- **Scalability Testing:** While designed with standard practices, the system is not load-tested for high-concurrency production scenarios.

1.4. User Scope and Audience

The system caters to a specific set of users, each with distinct interaction patterns.

User Scope Analysis:

1. Anonymous Visitor (Guest):

- **Goal:** Product Discovery.
- **Permissions:** Read-only access to home page, product listing, and product details. Restricted from adding items to cart or checking out.

2. Registered Customer:

- **Goal:** Commerce Transaction.
- **Permissions:** Full access to browsing features. Authority to manage personal cart, place orders, and view order history. Authenticated via JWT.

3. Administrator:

- **Goal:** User and System Management.
- **Permissions:** Manage user accounts, access system logs, and generate business reports.

4. Seller:

- **Goal:** Catalog and Promotion Management.
- **Permissions:** Manage products, categories, and coupons.

Target Audience: While simulating a commercial application, the primary consumers of this software and report are:

- **Course Lecturers:** To assess the team's ability to implement a full-stack solution.
- **Peer Students:** To serve as a reference implementation for future cohorts studying Web Application Development.

1.5. Methodology

The project followed a modified Waterfall methodology, adapted for a small team with a fixed deadline. The development lifecycle was divided into distinct phases:

1. Analysis Phase:

- **Requirement Gathering:** Listing core features (Login, Product, Cart).
- **Use Case Modeling:** Identifying actors (Admin, User) and their interactions.

2. Design Phase:

- **Database Schema:** Designing the Entity-Relationship Diagram (ERD) for Postgres, defining tables for `User`, `Product`, `Order`, `OrderItem`.
- **API Contract:** Defining RESTful endpoints (e.g., `GET /api/products`, `POST /api/cart`).
- **Wireframing:** Creating basic UI layouts for the React frontend.

3. Implementation Phase:

- **Backend First:** Built the Express server, Prisma models, and core logic. Tested via Postman.
- **Frontend Integration:** Built React components to consume the API.
- **Containerization:** Added Dockerfiles to ensure the app runs identically on all developer machines.

4. Testing Phase:

- **Unit Testing:** Basic tests for utility functions.
- **Manual Acceptance Testing:** Verifying the end-to-end user flow (Register -> Shop -> Checkout).

Tools & Environment:

- **IDE:** Visual Studio Code.
- **VCS:** Git (Branching strategy: Gitflow Workflow).
- **Project Management:** Basic Kanban board for tracking progress.

1.6. Report Structure

This report documents the entire lifecycle of “The Everything Shop” project. It is organized into 8 chapters:

- **Chapter 1: Introduction** – Outlines the project's necessity, objectives, scope, and the methodology used for development.
- **Chapter 2: Theoretical Basis and Technology** – Provides a deep dive into the technologies chosen (React, Express, Prisma, Docker) and the rationale behind them.
- **Chapter 3: System Requirements Analysis** – Details the functional specifications, actor interactions, and use case diagrams.
- **Chapter 4: System Design** – Presents the technical blueprints: Application Architecture, Database Design (ERD), and API Design.
- **Chapter 5: Implementation and Deployment** – Describes the actual coding structure, project organization, and how the system is deployed using Docker.
- **Chapter 6: Testing and Evaluation** – Covers the testing strategy, test cases executed, and an evaluation of the system against initial objectives.

- **Chapter 7: Conclusion and Future Development** – Summarizes the project's success, discusses lessons learned, and proposes future enhancements.
- **Chapter 8: References** – Lists all academic and technical resources cited.

2. THEORETICAL BASIS AND TECHNOLOGY

2.1. Theoretical Basis

This chapter outlines the theoretical frameworks and architectural patterns that underpin the design of the “Everything Shop”.

2.1.1. Model-View-Controller (MVC) Pattern

The application is structured using the **MVC architecture**, a design pattern that decouples the user interface (View), data (Model), and application logic (Controller).

- **Model:** Represents the data and business rules. In our system, this is managed by the **Prisma ORM** and the PostgreSQL database, wrapped by a **Service Layer** to handle business logic.
- **View:** The presentation layer that interacts with the user. Our **React** frontend serves as the view, consuming data from the API and rendering the UI.
- **Controller:** The interface between the Model and View. Our **Express** controllers receive HTTP requests, validate input, invoke the Service layer, and return the appropriate JSON response.

2.1.2. RESTful API

Communication between the frontend and backend relies on **REST (Representational State Transfer)** principles.

- **Statelessness:** Each request from the client to the server contains all the information needed to understand and process the request.
- **Resource-Based:** Data is exposed as resources (e.g., `/products`, `/orders`) manipulated using standard HTTP methods (GET, POST, PUT, DELETE).

2.1.3. Single Page Application (SPA)

The frontend is built as an SPA, meaning the application loads a single HTML page and dynamically updates content as the user interacts with the app. This provides a smoother user experience similar to a desktop application, avoiding full page reloads.

2.2. Overview of Related Technologies

The following technologies were selected to implement the system:

2.2.1. Frontend

- **React.js:** A JavaScript library for building user interfaces based on components. It allows for the creation of reusable UI elements and efficient state management.
- **Redux:** A predictable state container for JavaScript apps, used here to manage global states like user authentication status and shopping cart contents.

2.2.2. Backend

- **Node.js:** A JavaScript runtime built on Chrome’s V8 engine, allowing us to execute JavaScript on the server side.
- **Express.js:** A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.
- **Prisma:** A next-generation Object-Relational Mapper (ORM) for Node.js and TypeScript. It is used to define the data schema and interact with the database in a type-safe manner.

2.2.3. Database

- **PostgreSQL:** A powerful, open-source object-relational database system known for its reliability, feature robustness, and performance.

2.2.4. DevOps & Infrastructure

- **Docker:** A platform for developing, shipping, and running applications in containers. It ensures consistency across different development environments.
- **MinIO:** A high-performance, S3-compatible object storage server used for storing product images.
- **MailHog:** A developer tool for testing email sending capability without requiring a real SMTP server.

2.3. Rationale for Technology Selection

The selection of technologies was driven by the need for a modern, scalable, and developer-friendly stack.

2.3.1. Why PERN Stack (Postgres, Express, React, Node)?

- **Unified Language:** Using JavaScript/TypeScript on both the frontend and backend reduces context switching and allows for code reuse (e.g., shared types).
- **Ecosystem:** The Node.js ecosystem (npm) is the largest in the world, providing libraries for almost any functionality required (e.g., authentication, validation).
- **Performance:** Node.js's non-blocking I/O model is well-suited for I/O-heavy applications like e-commerce APIs.

2.3.2. Why TypeScript?

We chose TypeScript over plain JavaScript to enforce static typing. This helps catch errors at compile-time rather than runtime, significantly improving code quality and maintainability, especially in the backend logic.

2.3.3. Why Containerization?

Using Docker and Docker Compose solves the “it works on my machine” problem. By defining the environment (Node version, Database version) in code, every team member can spin up the entire stack with a single command, ensuring a consistent development experience.

3. SYSTEM REQUIREMENTS ANALYSIS

3.1. Problem Description and Business Logic

3.1.1. Problem Description

The rapid growth of e-commerce has created a need for versatile platforms that simulate real-world complexities for educational purposes. “The Everything Shop” addresses the challenge of building a full-stack system that integrates multiple distinct modules—User Management, Product Catalog, Ordering, Payment Simulation, and Reporting—into a cohesive unit. The system aims to solve the problem of fragmented learning by providing a unified environment where students can experience the lifecycle of a commercial application, from inventory management by sellers to the checkout process by customers.

3.1.2. Business Logic Analysis

The core business flow follows a standard e-commerce cycle:

1. **Product Listing:** Sellers and Admins populate the system with products (UC-14), categorized hierarchically (UC-18).
2. **Product Discovery:** Guests and Customers browse (UC-24) or search (UC-26) for these products.
3. **Order Creation:** A Customer adds items to a cart (UC-29) and proceeds to checkout, creating an Order (UC-30). This process **includes** Payment Initiation (UC-37) and optionally **extends** to Coupon Application (UC-42).
4. **Order Fulfillment:** Sellers manage the order status (UC-32), potentially updating delivery info (UC-35).
5. **Post-Purchase:** Customers can view history (UC-31), request returns (UC-46), or receive notifications for back-in-stock items (UC-23).
6. **Reporting:** The system aggregates this data into reports (UC-51) for business analysis.

3.2. Stakeholders

The system involves several key actors, each with defined roles and responsibilities:

1. **Guest** (Unregistered User)
 - **Role:** Potential customer exploring the platform.
 - **Capabilities:** Can browse categories, search for products, view product details, and register for an account. Cannot purchase or manage a cart.
2. **Customer** (Registered User)
 - **Role:** End-user purchasing goods.
 - **Capabilities:** Inherits Guest capabilities. Can manage profile, maintain a persistent shopping cart, place orders, make payments (simulated), review order history, and request returns.
3. **Seller** (Vendor)
 - **Role:** Merchant managing their own inventory and sales.
 - **Capabilities:** Create and manage own products, view orders related to their products, update order status (e.g., Shipping), and view basic performance reports.
4. **Administrator** (System Owner)
 - **Role:** Supervisor of the entire platform.
 - **Capabilities:** Full access to all modules. Manages user accounts (ban/un-ban), defines global product categories, oversees all orders, and accesses comprehensive system-wide reports and audit logs.

5. System Actors (Automated Services)

- **Payment Gateway (Mock):** simulating financial transactions.
- **Notification Service:** Sending emails (via MailHog) for order confirmations or stock alerts.
- **Logging Service:** Recording audit trails for security and debugging.

3.3. Functional Requirements

The functional requirements are categorized by module, derived from the use case analysis.

3.3.1. User Management (EP-01)

- **UC-01 Register:** Allow guests to create accounts.
- **UC-02 Login:** Authenticate via Email/Password.
- **UC-08 RBAC:** Enforce strict role checks (Admin vs Seller vs Customer).
- **UC-11 View Users:** Admins can list and manage all user accounts.

3.3.2. Product Management (EP-02)

- **UC-14 Create Product:** Sellers can add new items. **Includes** Media Upload (UC-17) for product images.
- **UC-18 Categories:** Admins define multi-level category trees.
- **UC-22 Stock Alerts:** System notifies sellers when stock is low.

3.3.3. Shopping & Ordering (EP-03, EP-04)

- **UC-24 Browse:** Users can filter products by category or attributes.
- **UC-30 Create Order:** Customers convert cart to order. **Includes** Payment (UC-37). **Extended by** Coupon Application (UC-42).
- **UC-32 Manage Order:** Sellers update status (Pending -> Delivering -> Delivered).

3.3.4. Promotions & Returns (EP-06, EP-07)

- **UC-41 Coupons:** Sellers create validation rules for discounts.
- **UC-46 Returns:** Customers can request refunds for delivered items.

3.3.5. Reporting (EP-08)

- **UC-54 Dashboard:** Visual summary of sales.
- **UC-53 Export:** **Extends** report generation to allow downloading CSV/PDF.

3.4. Non-Functional Requirements

3.4.1. Performance Requirements

- **Response Time:** The system should respond to user interactions (e.g., page loads, API calls) within **10 seconds**. This relaxed constraint accounts for the limitations of the on-premise hosting environment used for this student project.
- **Throughput:** Capable of handling concurrent access by a small cohort of users (e.g., class demonstration).

3.4.2. internal/Reliability & Availability

- **Uptime:** The target availability is **95%** during evaluation periods. Occasional downtime is expected due to the non-redundant infrastructure.
- **Data Integrity:** Database transactions must support ACID properties to ensure order data is never lost or corrupted.

3.4.3. Security Requirements

- **Authentication:** All protected resources require a valid JSON Web Token (JWT).
- **Password Storage:** Passwords must be hashed using robust algorithms (e.g., bcrypt) before storage.
- **Authorization:** API endpoints must validate the user's role (Admin, Seller, Customer) before processing requests.

3.4.4. Usability

- **Interface:** The web interface should be responsive and accessible on standard desktop browsers.
- **Feedback:** Clear error messages and loading states should be provided to the user.

3.5. Use Case Diagram

3.5.1. Use Case Diagrams

The system's use cases are divided into 8 Epics for clarity.

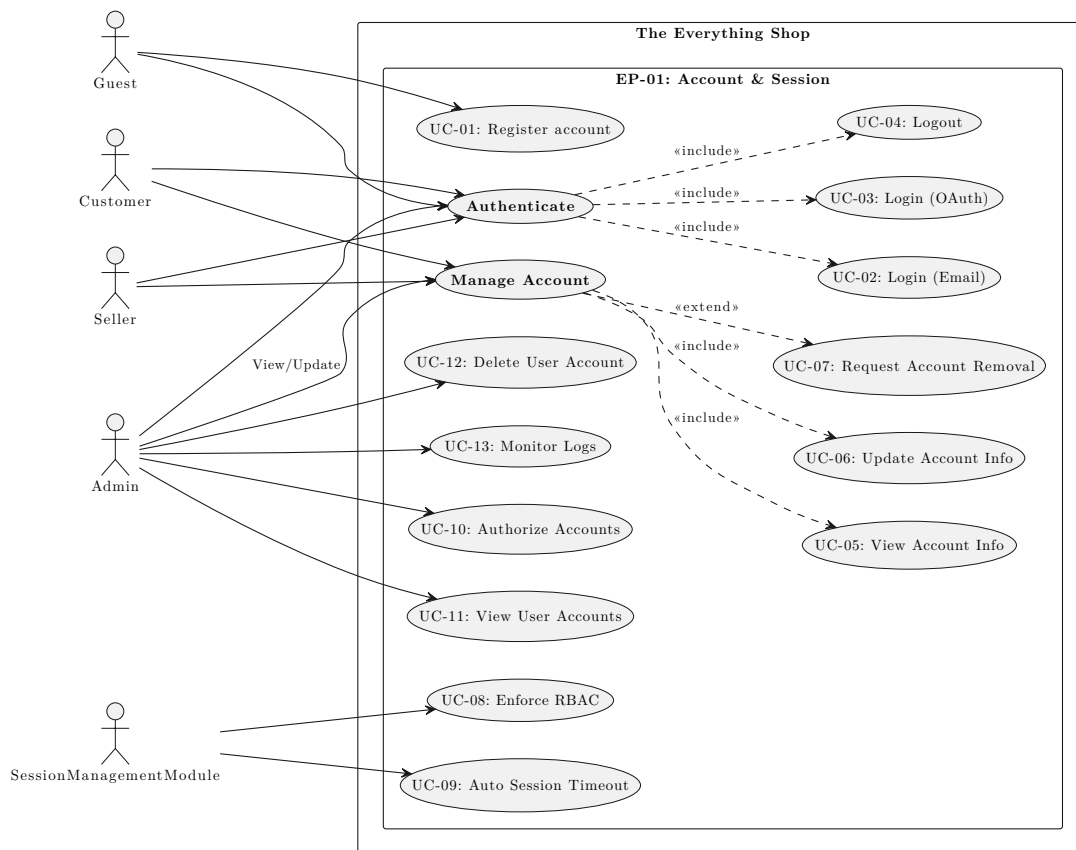


Figure 1: Epic 01: Account & Session

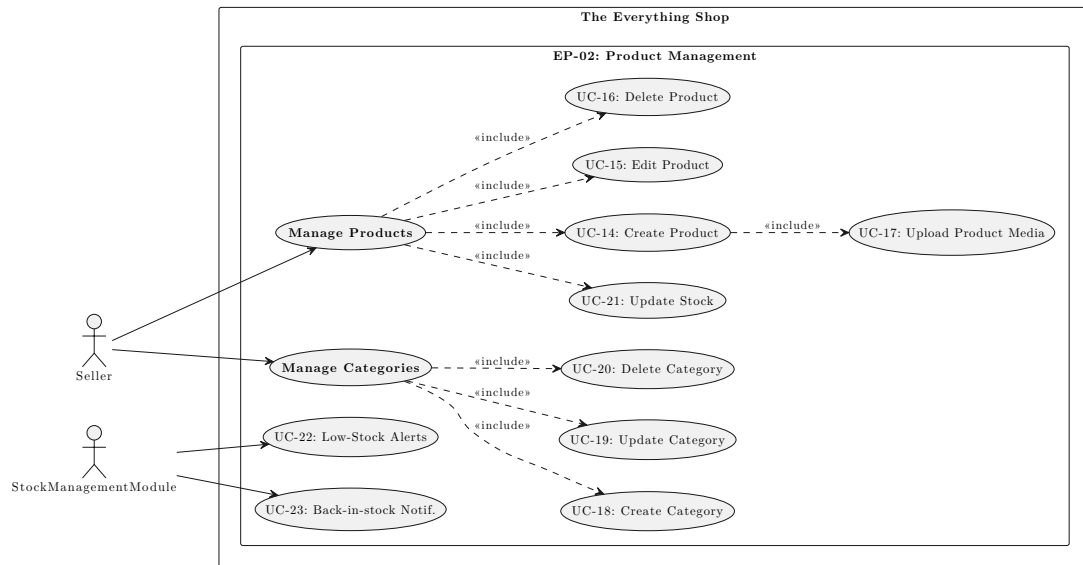


Figure 2: Epic 02: Product Management

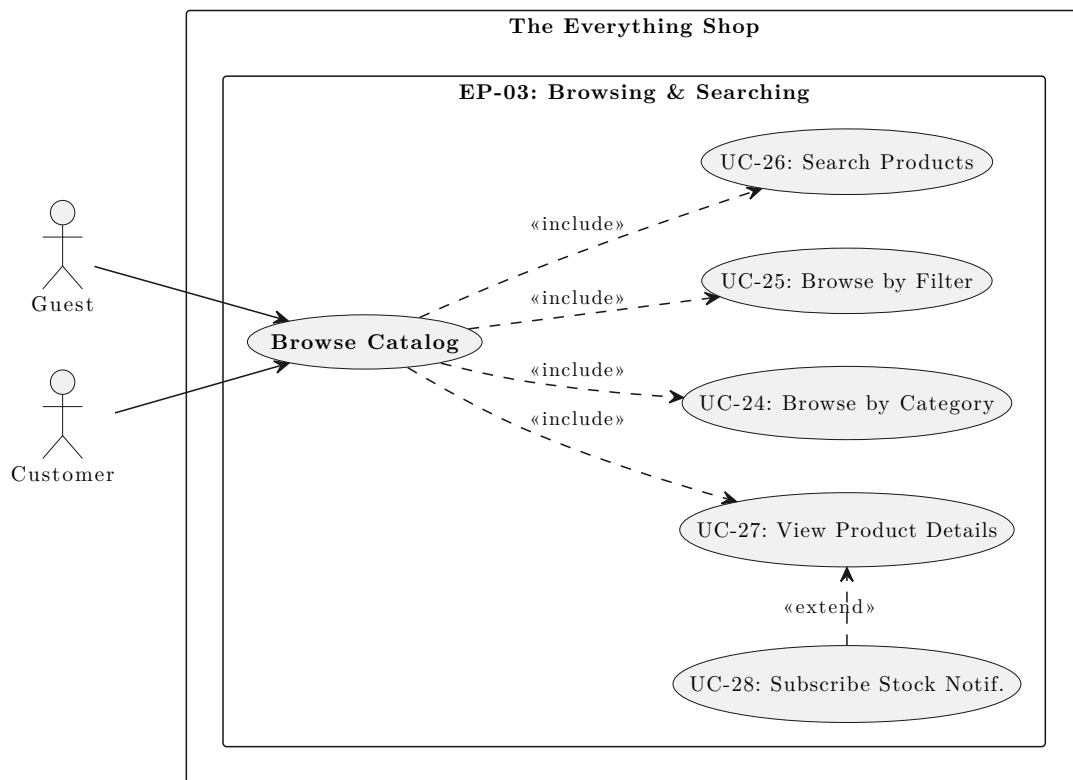


Figure 3: Epic 03: Browsing & Searching

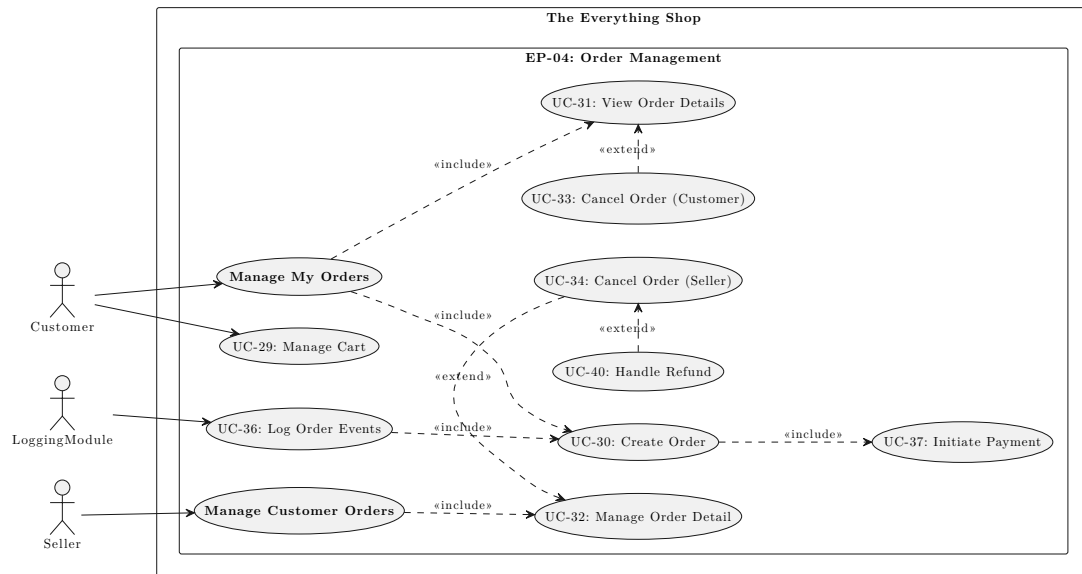


Figure 4: Epic 04: Order Management

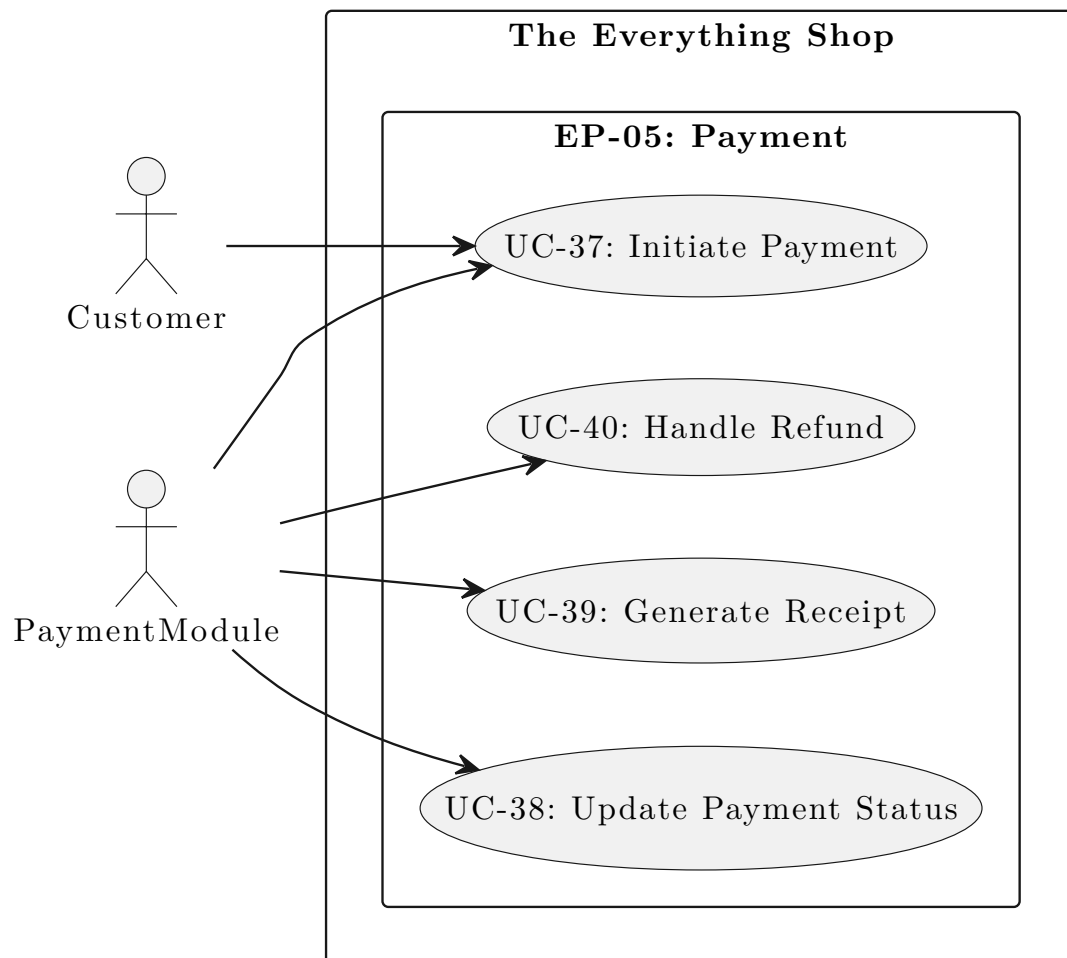


Figure 5: Epic 05: Payment

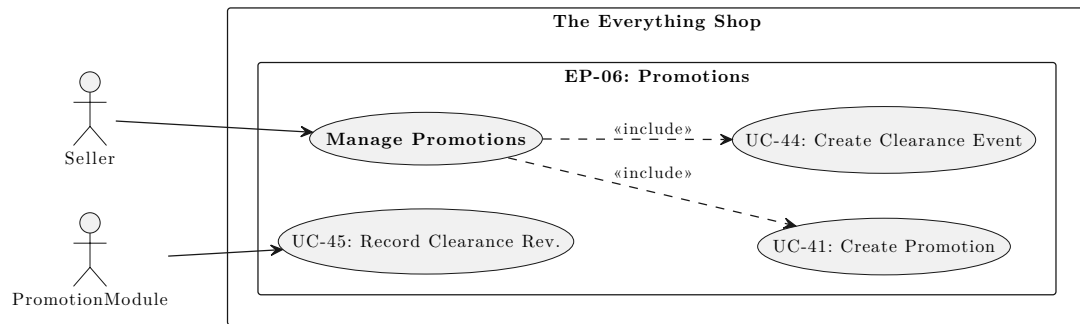


Figure 6: Epic 06: Promotions

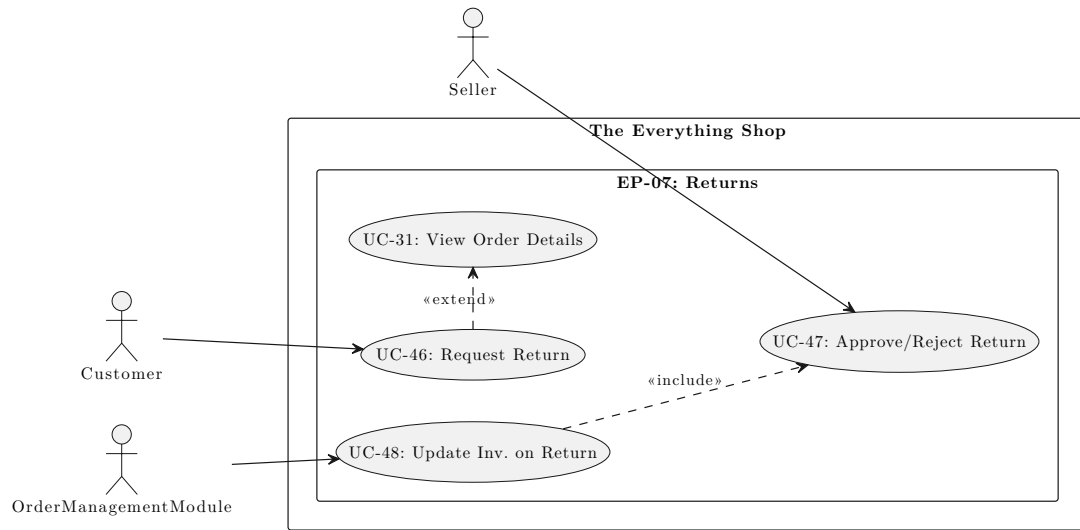


Figure 7: Epic 07: Returns

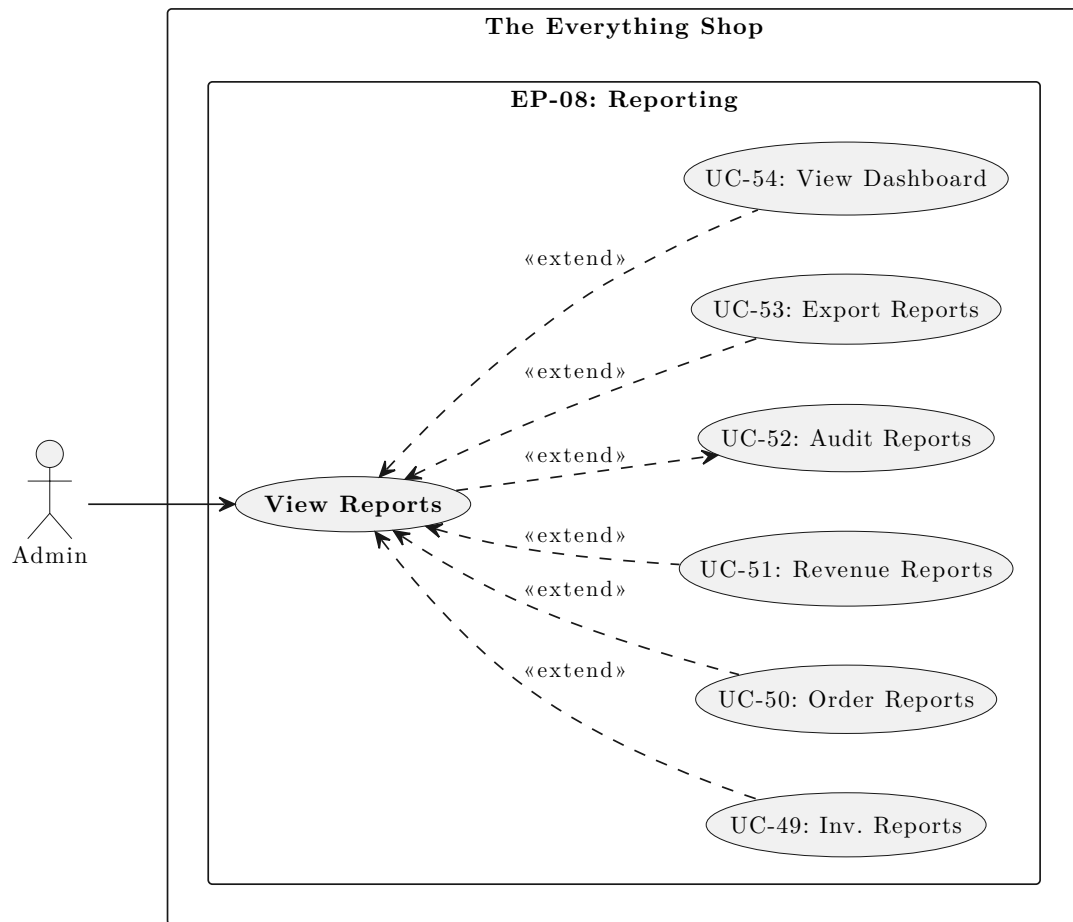


Figure 8: Epic 08: Reporting

3.6. Use Case Specifications

This section provides detailed specifications for the most critical use cases in the system.

3.6.1. User Management

Use Case ID	UC-01
Use Case Name	Register Account
Description	Allows a new visitor to create a customer account to access personalized features.
Actors	Guest
Priority	High
Trigger	Guest selects “Register” option on the homepage.
Pre-conditions	Guest is not currently logged in.
Basic Flow	1. System displays the Registration Form. 2. Guest enters name, email, password, and confirmation. 3. System validates the input format and checks email uniqueness. 4. System creates a new Customer record. 5. System displays a success message and redirects to Login.
Alternative Flows	• Invalid Input: If validation fails, System highlights errors and requests correction.
Exception Flows	• Email Taken: If email exists, System notifies user and suggests recovery.
Post-conditions	New Customer account is created in the database.

Table 1: Specification for UC-01: Register Account

Use Case ID	UC-02
Use Case Name	Login (Email)
Description	Allows users to authenticate and access their account.
Actors	Guest, Customer, Seller, Admin
Priority	High
Trigger	Guest submits login credentials.
Pre-conditions	User must have a registered account.
Basic Flow	1. Guest enters email and password. 2. System validates credentials against the database. 3. System generates a JWT token. 4. System redirects user to the homepage/dashboard.
Alternative Flows	-
Exception Flows	• Auth Failed: If credentials do not match, System displays “Invalid email or password”.
Post-conditions	User is authenticated and session is active.

Table 2: Specification for UC-02: Login

Use Case ID	UC-05
Use Case Name	View Account Information
Description	Displays the user's personal profile content.
Actors	Customer, Seller, Admin
Priority	Low
Trigger	User navigates to "My Profile".
Pre-conditions	User is logged in.
Basic Flow	1. System retrieves user details from the database. 2. System displays name, email, and role.
Alternative Flows	-
Exception Flows	-
Post-conditions	User profile is displayed.

Table 3: Specification for UC-05: View Account Information

Use Case ID	UC-06
Use Case Name	Update Account Information
Description	Allows users to modify their personal details.
Actors	Customer, Seller, Admin
Priority	Medium
Trigger	User clicks "Edit Profile".
Pre-conditions	User is logged in.
Basic Flow	1. User modifies name or other allowed fields. 2. User submits changes. 3. System validates input. 4. System updates the database. 5. System shows success message.
Alternative Flows	-
Exception Flows	• Update Failed: Database error triggers an alert to the user.
Post-conditions	User record is updated.

Table 4: Specification for UC-06: Update Account Information

3.6.2. Product Management

Use Case ID	UC-14
Use Case Name	Create Product
Description	Enables Sellers to list a new item for sale.
Actors	Seller
Priority	High
Trigger	Seller clicks “Add New Product”.
Pre-conditions	Seller is logged in.
Basic Flow	1. System displays Product Creation form. 2. Seller enters name, price, stock, and category. 3. System invokes UC-17 Upload Media. 4. Seller submits the form. 5. System saves product to database.
Alternative Flows	-
Exception Flows	• Validation Error: Missing fields prevent submission.
Post-conditions	Product is visible in the catalog.

Table 5: Specification for UC-14: Create Product

Use Case ID	UC-15
Use Case Name	Edit Product
Description	Allows Sellers to update product details.
Actors	Seller
Priority	Medium
Trigger	Seller selects a product to edit.
Pre-conditions	Product exists and belongs to Seller.
Basic Flow	1. System pre-fills form with current data. 2. Seller modifies fields (e.g., description, price). 3. Seller saves changes. 4. System updates the record.
Alternative Flows	-
Exception Flows	-
Post-conditions	Product information is updated.

Table 6: Specification for UC-15: Edit Product

Use Case ID	UC-17
Use Case Name	Upload Product Media
Description	Handles image file uploading and storage.
Actors	Seller
Priority	Medium
Trigger	Included by UC-14 or UC-15.
Pre-conditions	Valid image file format.
Basic Flow	1. Seller selects image file(s). 2. System validates file type and size. 3. System uploads to Object Storage (MinIO). 4. Storage returns a public URL. 5. System attaches URL to the product form.
Alternative Flows	-
Exception Flows	• Upload Error: Storage unavailable/quota exceeded.
Post-conditions	Image URL is ready to be saved.

Table 7: Specification for UC-17: Upload Product Media

Use Case ID	UC-21
Use Case Name	Update Stock Quantity
Description	Adjusts inventory levels.
Actors	Seller
Priority	High
Trigger	Seller manually updates stock or Order placement occurs.
Pre-conditions	Product exists.
Basic Flow	1. Seller inputs new quantity. 2. System updates inventory count. 3. System checks for low-stock triggers.
Alternative Flows	-
Exception Flows	-
Post-conditions	Inventory reflects new quantity.

Table 8: Specification for UC-21: Update Stock Quantity

3.6.3. Shopping & Browsing

Use Case ID	UC-24
Use Case Name	Browse Products by Category
Description	Allows users to navigate products via hierarchy.
Actors	Guest, Customer
Priority	High
Trigger	User selects a category from menu.
Pre-conditions	-
Basic Flow	1. System retrieves products linked to the selected category. 2. System displays products in a grid.
Alternative Flows	• Filtering: User applies additional filters to the category view.
Exception Flows	• No Products: System indicates category is empty.
Post-conditions	Products are displayed.

Table 9: Specification for UC-24: Browse Products by Category

Use Case ID	UC-26
Use Case Name	Search Products
Description	Allows users to find products by keywords.
Actors	Guest, Customer
Priority	High
Trigger	User enters text in search bar.
Pre-conditions	-
Basic Flow	1. User executes search query. 2. System matches query against product names/descriptions. 3. System returns matching results.
Alternative Flows	-
Exception Flows	• No Matches: System suggests broadening search.
Post-conditions	Search results are displayed.

Table 10: Specification for UC-26: Search Products

Use Case ID	UC-27
Use Case Name	View Product Details
Description	Shows comprehensive information about a specific item.
Actors	Guest, Customer
Priority	High
Trigger	User clicks a product card.
Pre-conditions	Product exists.
Basic Flow	1. System fetches full product details (variants, images, reviews). 2. System renders the Product Detail Page.
Alternative Flows	-
Exception Flows	• Deleted Product: Redirects to 404/Catalog.
Post-conditions	User views details.

Table 11: Specification for UC-27: View Product Details

3.6.4. Ordering Process

Use Case ID	UC-29
Use Case Name	Create & Update Shopping Cart
Description	Manages temporary selection of products.
Actors	Customer
Priority	High
Trigger	Customer clicks “Add to Cart”.
Pre-conditions	Customer is logged in, Product has stock.
Basic Flow	1. System adds item to user’s persistent cart. 2. User views cart. 3. User can modify quantities or remove items. 4. System updates cart totals.
Alternative Flows	-
Exception Flows	• Out of Stock: User tries to add more than available; System restricts action.
Post-conditions	Cart state is updated.

Table 12: Specification for UC-29: Create & Update Shopping Cart

Use Case ID	UC-30
Use Case Name	Create Order
Description	Finalizes the purchase transaction.
Actors	Customer
Priority	Critical
Trigger	Customer clicks “Checkout” from Cart.
Pre-conditions	Cart is not empty.
Basic Flow	1. System validates stock for all cart items. 2. Customer confirms delivery address. 3. Customer optionally applies Coupon (UC-42). 4. System calculates final total. 5. Customer proceeds to Payment (UC-37). 6. System creates Order record with status “Pending”. 7. System clears the Cart.
Alternative Flows	-
Exception Flows	• Payment Failed: Order creation aborts, user returned to checkout. • Stock Change: Items sold out during checkout process alert user.
Post-conditions	Order is placed, Inventory reduced (UC-21).

Table 13: Specification for UC-30: Create Order

Use Case ID	UC-31
Use Case Name	View Order Details
Description	Allows customers to track their purchase.
Actors	Customer
Priority	Medium
Trigger	Customer selects an order from history.
Pre-conditions	Order belongs to Customer.
Basic Flow	1. System retrieves order items, status, and timeline. 2. System displays details.
Alternative Flows	-
Exception Flows	-
Post-conditions	Order info is displayed.

Table 14: Specification for UC-31: View Order Details

3.6.5. Order Management

Use Case ID	UC-32
Use Case Name	Manage Order
Description	Sellers update the lifecycle of an order.
Actors	Seller
Priority	High
Trigger	Seller reviews pending orders.
Pre-conditions	Order contains Seller's products.
Basic Flow	1. Seller views order details. 2. Seller updates status (e.g., "Processing" -> "Delivering"). 3. System notifies Customer of status change.
Alternative Flows	-
Exception Flows	• Invalid Transition: Trying to 'ship' a cancelled order. System blocks.
Post-conditions	Order status is updated.

Table 15: Specification for UC-32: Manage Order

Use Case ID	UC-33
Use Case Name	Cancel Order (Customer)
Description	Allows customers to stop an order before processing.
Actors	Customer
Priority	Medium
Trigger	Customer clicks "Cancel" on an order.
Pre-conditions	Order status is "Pending".
Basic Flow	1. Customer confirms cancellation. 2. System refunds payment (UC-40). 3. System restocks inventory (UC-21). 4. System marks order as "Cancelled".
Alternative Flows	-
Exception Flows	• Too Late: Order is already "Processing"; Cancellation denied.
Post-conditions	Order is cancelled.

Table 16: Specification for UC-33: Cancel Order (Customer)

Use Case ID	UC-35
Use Case Name	Modify Delivery Info
Description	Corrects shipping details before dispatch.
Actors	Seller
Priority	Low
Trigger	Customer request or error discovery.
Pre-conditions	Order not yet “Delivering”.
Basic Flow	1. Seller edits shipping address/notes. 2. System saves changes.
Alternative Flows	-
Exception Flows	-
Post-conditions	Delivery info updated.

Table 17: Specification for UC-35: Modify Delivery Info

3.6.6. Promotions & Reporting

Use Case ID	UC-41
Use Case Name	Create/Manage Promotion
Description	Defines discount rules.
Actors	Seller
Priority	Medium
Trigger	Seller creates a coupon.
Pre-conditions	Seller is logged in.
Basic Flow	1. Seller defines code, discount %, and validity period. 2. System saves coupon.
Alternative Flows	-
Exception Flows	-
Post-conditions	Coupon is active.

Table 18: Specification for UC-41: Create/Manage Promotion

Use Case ID	UC-46
Use Case Name	Request Return
Description	Initiates the RMA process.
Actors	Customer
Priority	Medium
Trigger	Customer reports issue with delivered item.
Pre-conditions	Order is “Delivered”.
Basic Flow	1. Customer selects item and reason (Damaged/Wrong Item). 2. System creates Return Request. 3. Seller notified to Approve/Reject (UC-47).
Alternative Flows	-
Exception Flows	• Policy Expiry: Return window passed.
Post-conditions	Return Request is pending.

Table 19: Specification for UC-46: Request Return

4. SYSTEM DESIGN

4.1. Overall Architecture Design

4.1.1. Overall Architecture Design

The system follows a classic **3-Tier Architecture**, ensuring separation of concerns, scalability, and maintainability.

4.1.1.1. Presentation Layer (Frontend)

- **Technology:** React (Vite), Redux.
- **Responsibility:** Handles user interactions, renders UI components, and communicates with the backend via RESTful APIs.
- **Key Components:**
 - **Client App:** Single Page Application (SPA) served to the user's browser.
 - **Admin/Seller Dashboard:** Dedicated interfaces for privileged users.

4.1.1.2. Application Layer (Backend)

- **Technology:** Node.js, Express.js.
- **Responsibility:** Processes business logic, handles API requests, enforces security (Authentication/Authorization), and interacts with the database.
- **Key Components:**
 - **API Server:** Exposes REST endpoints.
 - **Middleware:** Handles logging, error handling, and authentication (JWT).
 - **Modules:** Auth, Product, Order, Payment, etc.

4.1.1.3. Data Layer (Database)

- **Technology:** PostgreSQL (Relational Database).
- **Responsibility:** Persists application data including users, products, orders, and transactions.
- **ORM:** Prisma is used for type-safe database access and schema management.

Overall System Architecture (3-Tier)

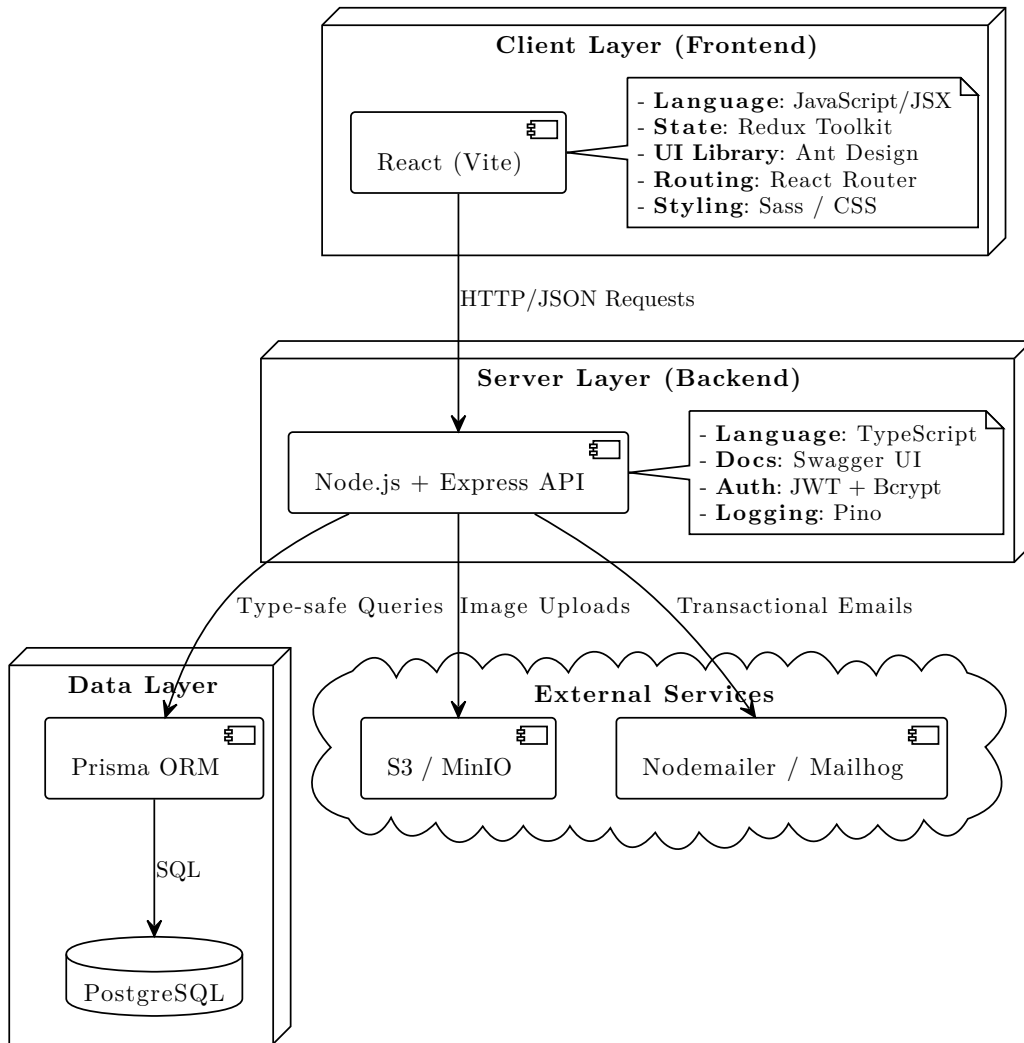


Figure 9: System Architecture Diagram

4.2. Functional Design

4.2.1. Functional Design and Module Details

The system is decomposed into core functional modules. Each module is documented with its responsibilities, key features, and detailed sequence diagrams illustrating critical workflows.

4.2.1.1. Authentication Module

- **Responsibilities:** Handles user registration, login, logout, and token management.
- **Key Features:**
 - Email/Password login.
 - JWT generation and refreshing (Access/Refresh tokens).
 - Role-Based Access Control (RBAC).

Detailed Flows:

1. **Registration:** User provides credentials -> Hashed (bcrypt) -> Stored in DB.
2. **Login:** User validates credentials -> Server issues JWTs.

3. **Access Control:** Middleware checks JWT in headers.

Auth: Registration Sequence

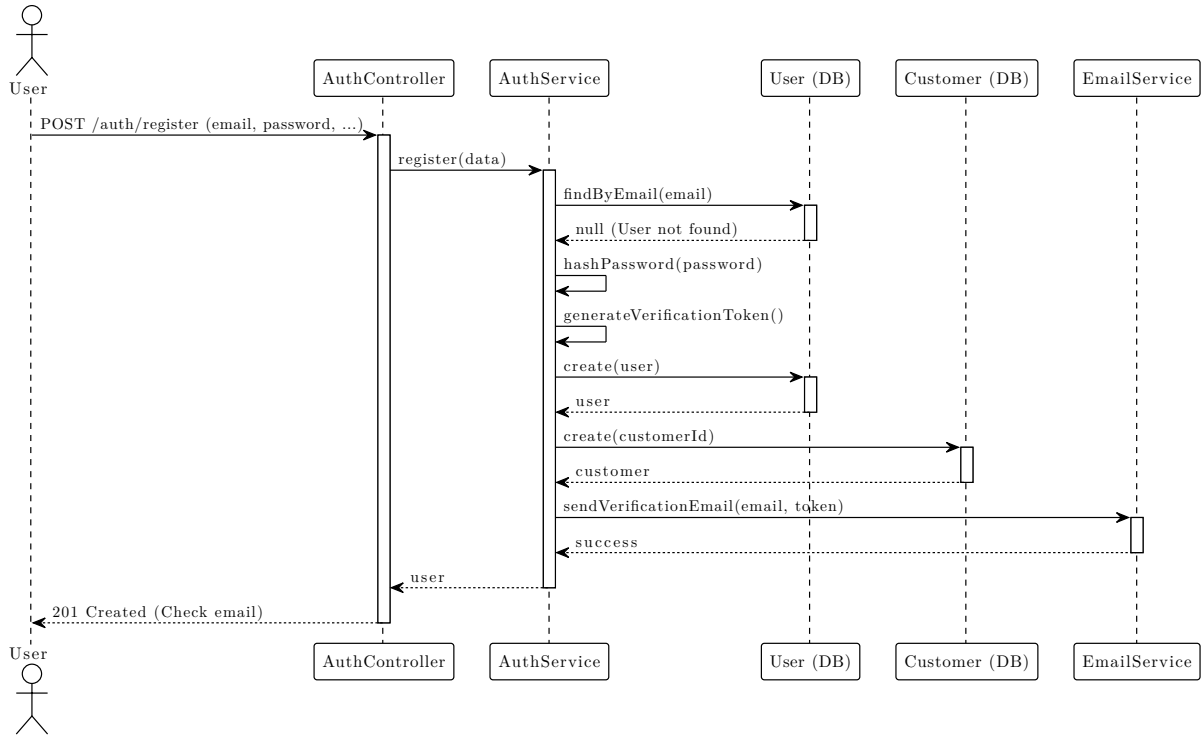


Figure 10: Registration Sequence

Auth: Login Sequence

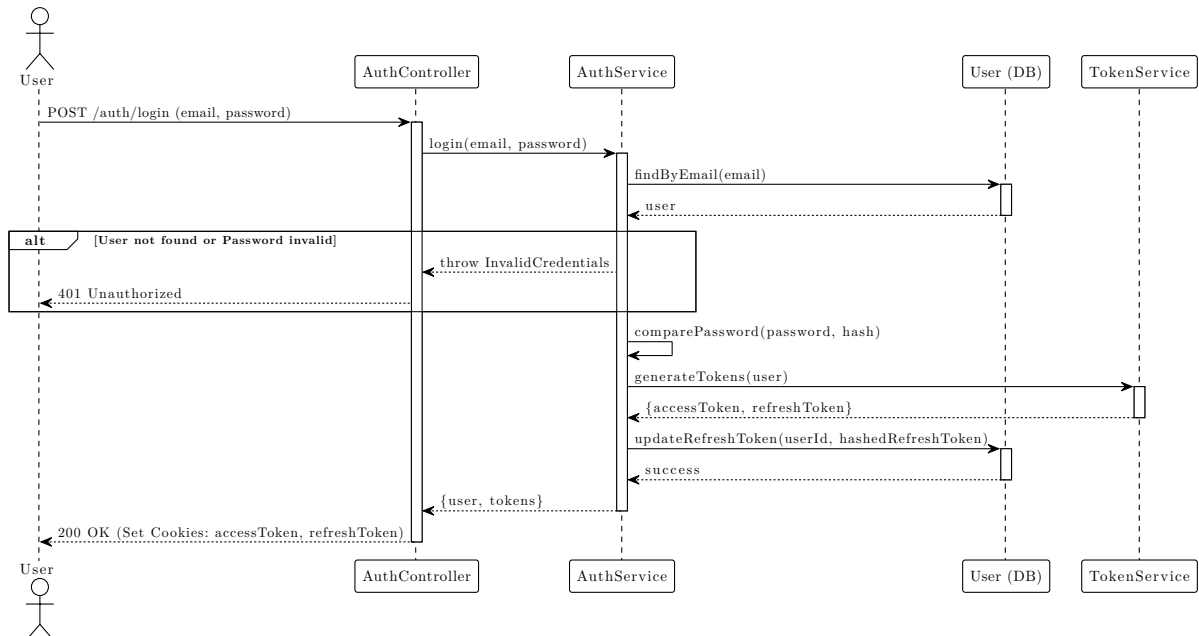


Figure 11: Login Sequence

Auth: Verify Email Sequence

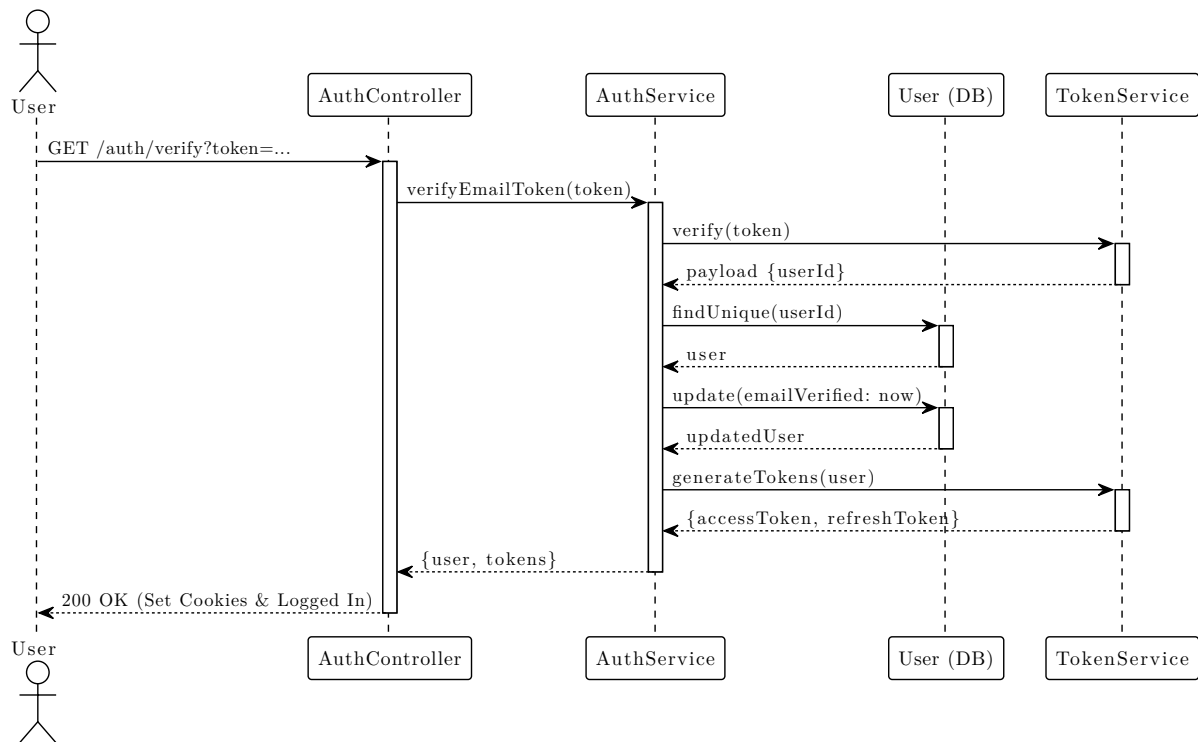


Figure 12: Verify Email Sequence

Auth: Forgot Password Sequence

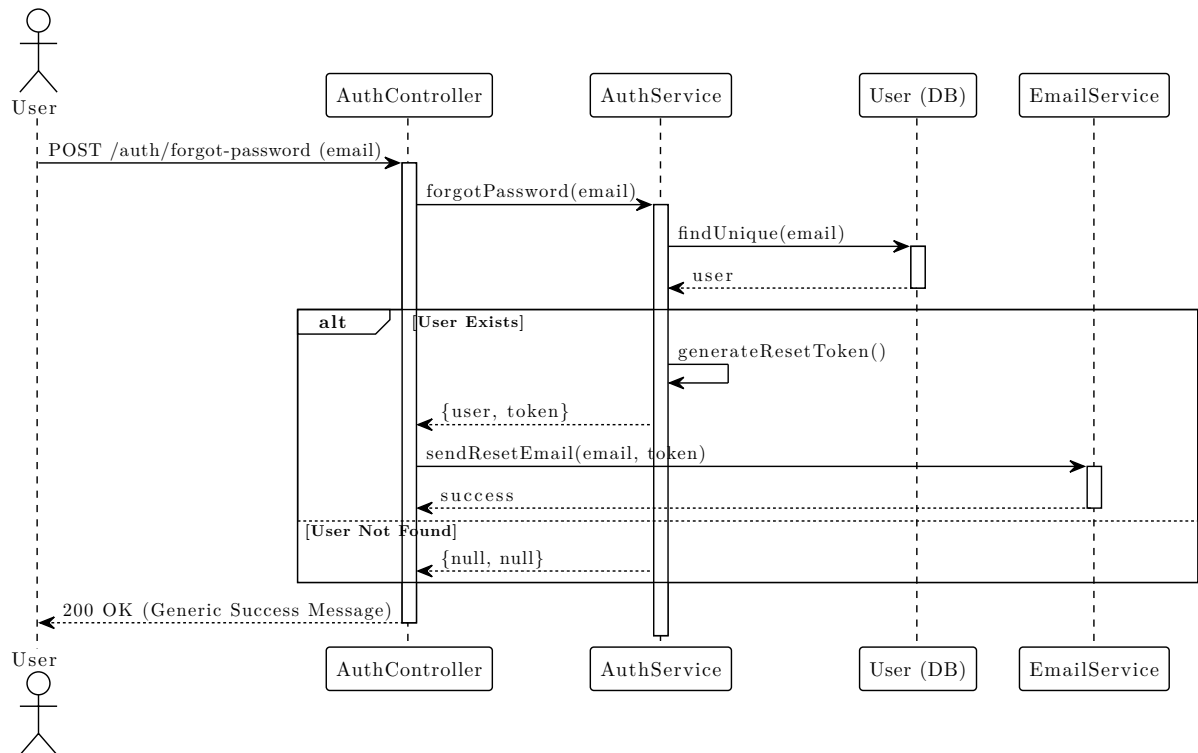


Figure 13: Forgot Password Sequence

Auth: Reset Password Sequence

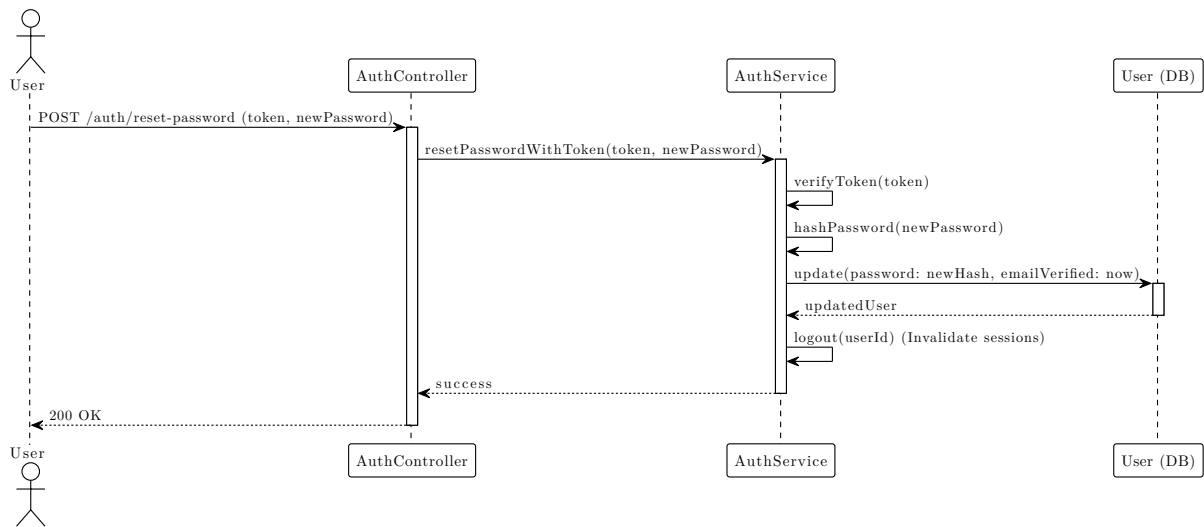


Figure 14: Reset Password Sequence

Auth: Token Refresh Sequence

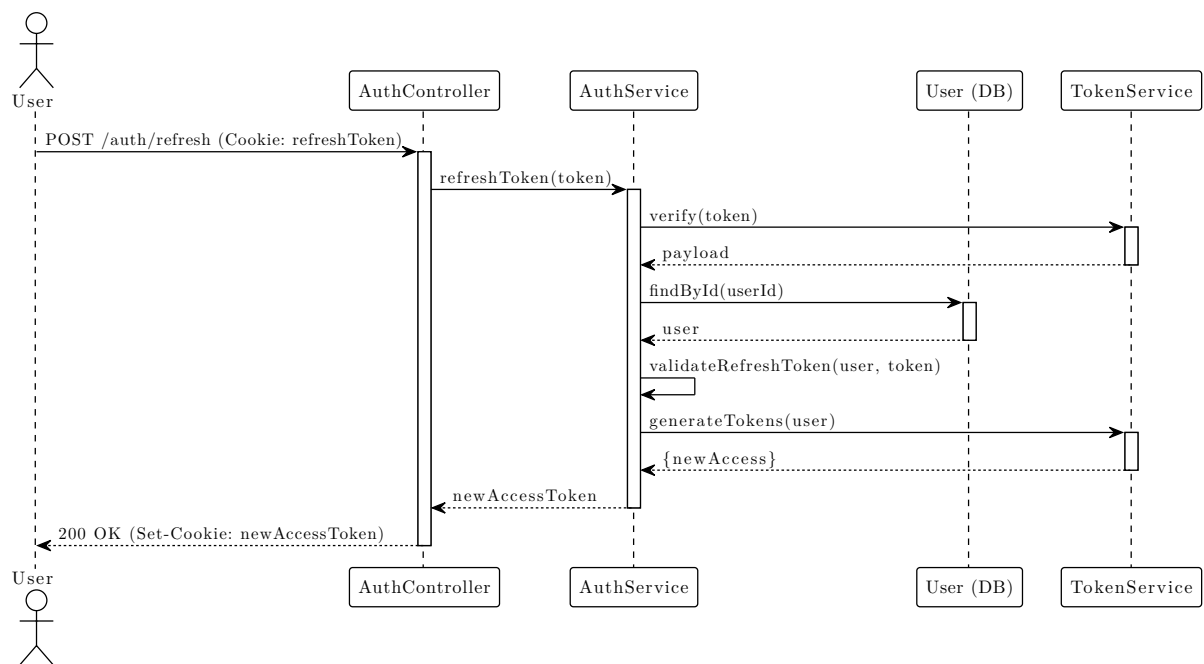


Figure 15: Token Refresh Sequence

Auth: Logout Sequence

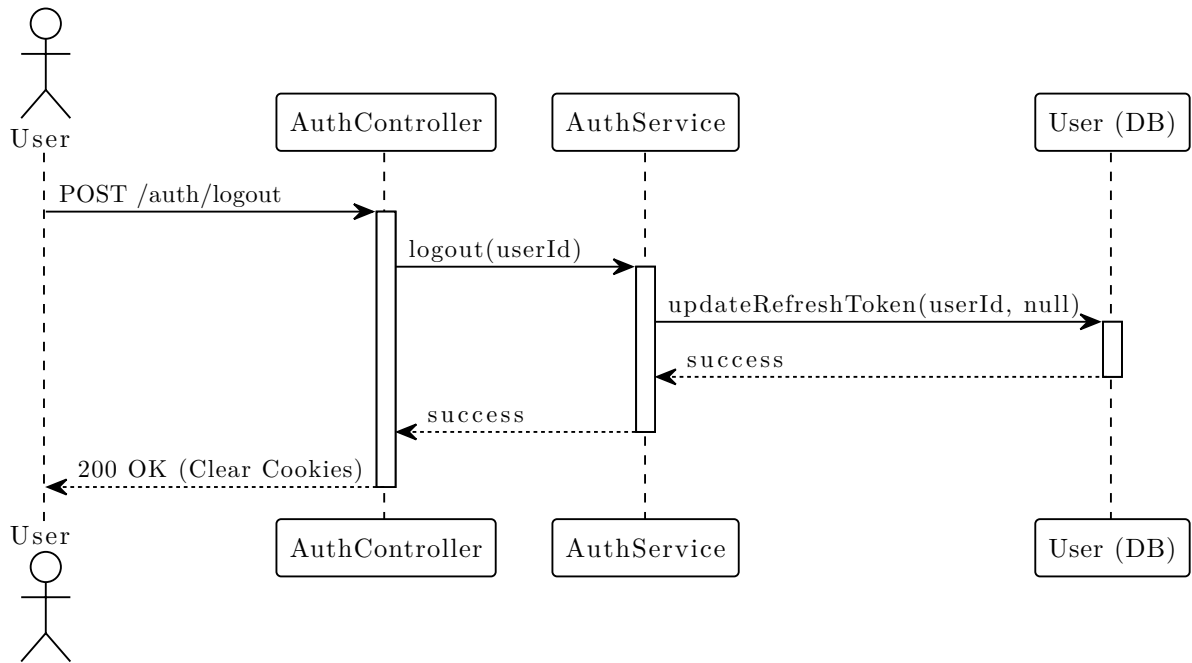


Figure 16: Logout Sequence

4.2.1.2. Product Management Module

- **Responsibilities:** Manages catalog, categories, and inventory.
- **Key Features:** CRUD operations, Image upload, Stock alerts.

Product: Create Product Sequence

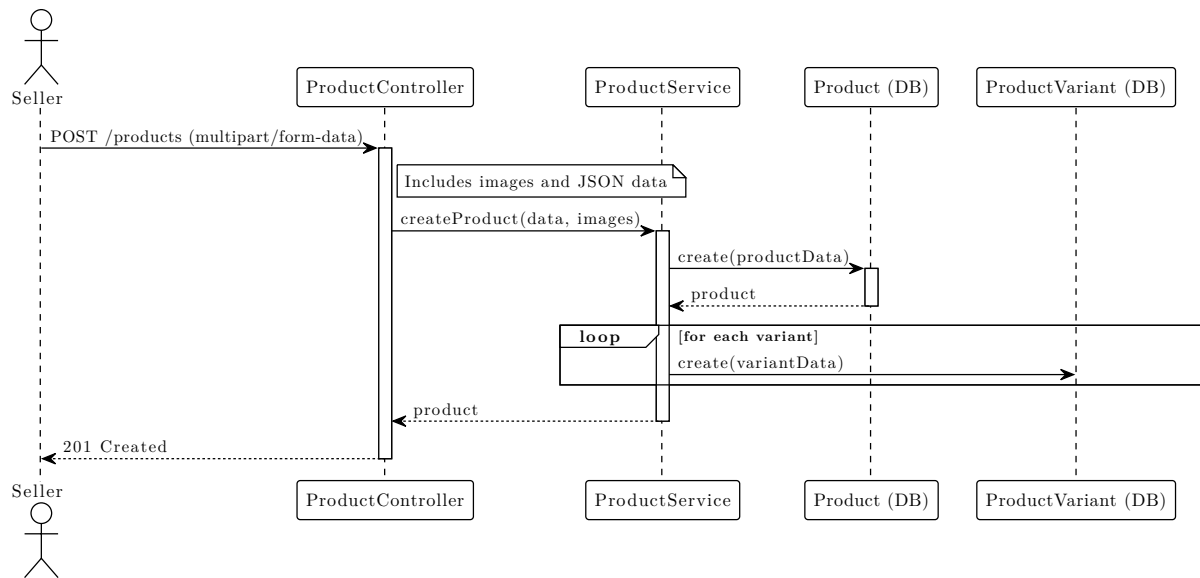


Figure 17: Create Product Sequence

Product: Search & Filter Sequence

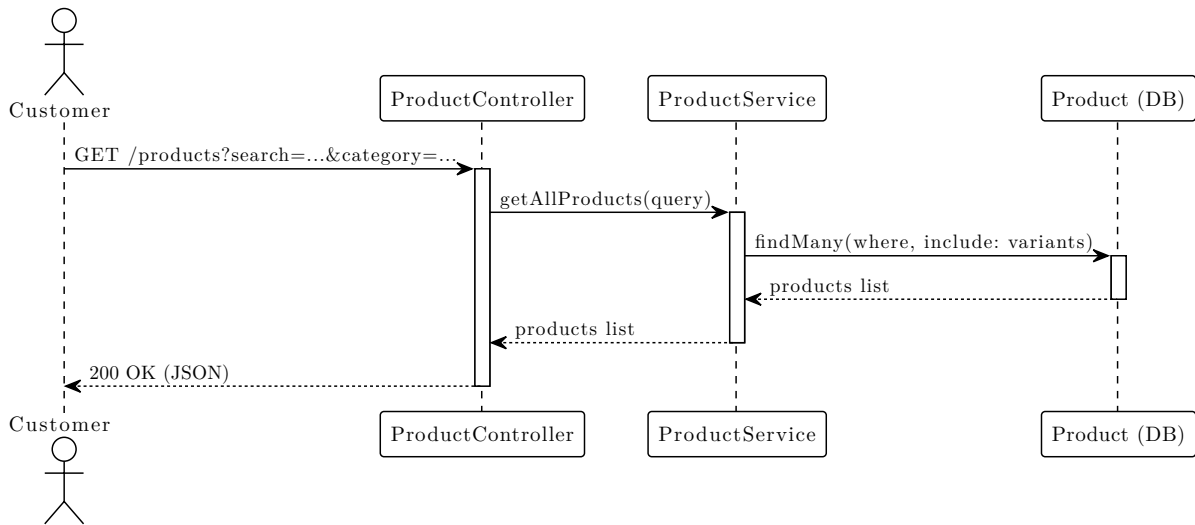


Figure 18: Search & Filter Sequence

Product: Update Stock Sequence

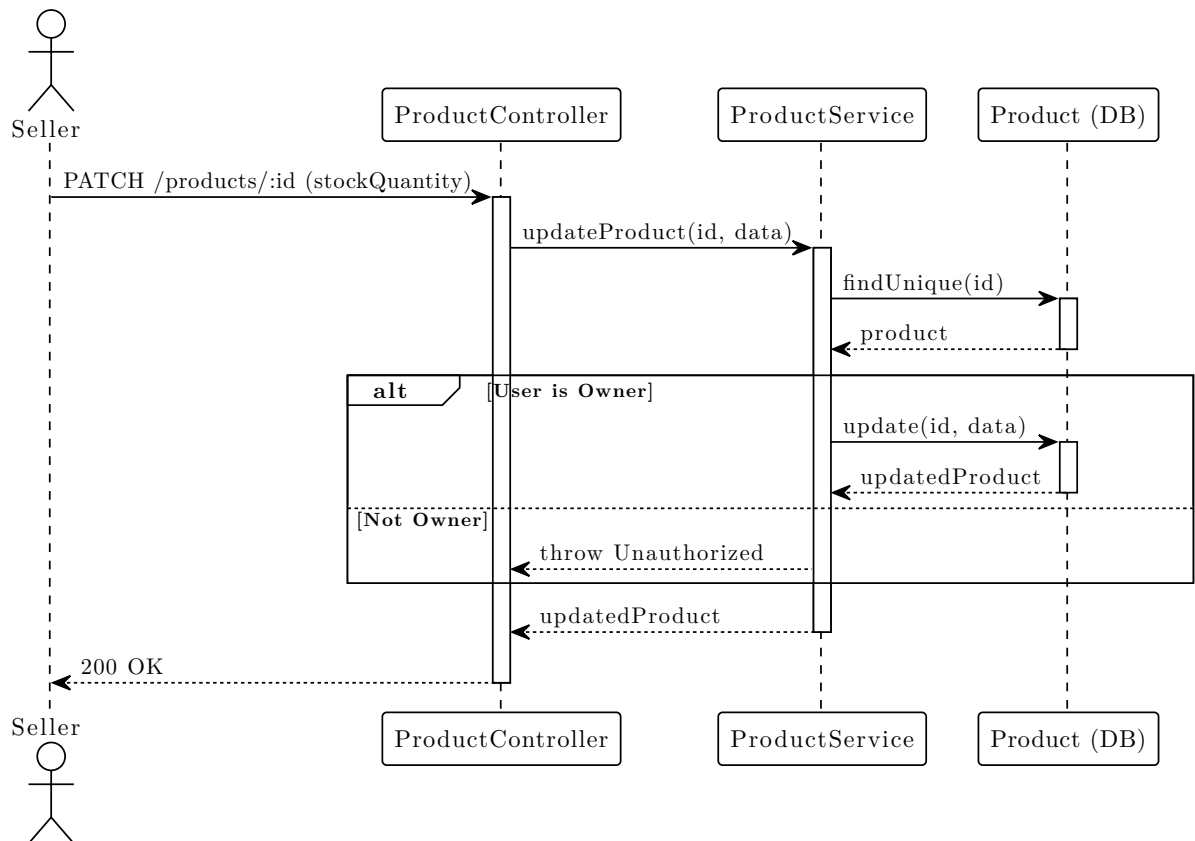


Figure 19: Update Stock Sequence

Product: Get Product Detail Sequence

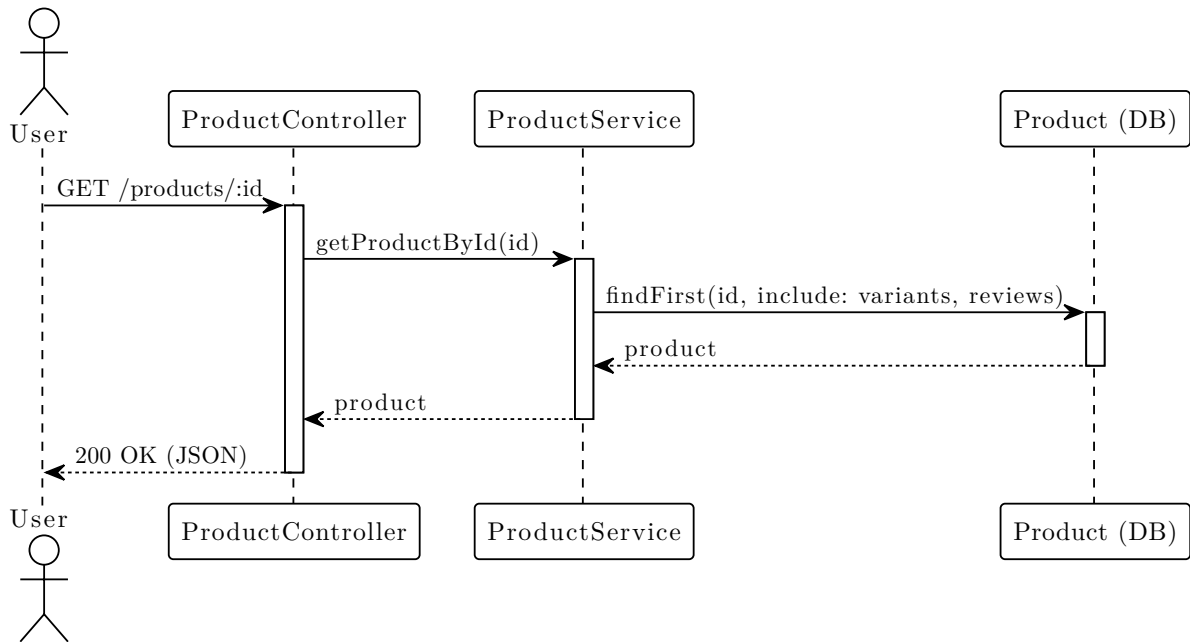


Figure 20: Get Product Detail Sequence

Product: Delete Product Sequence

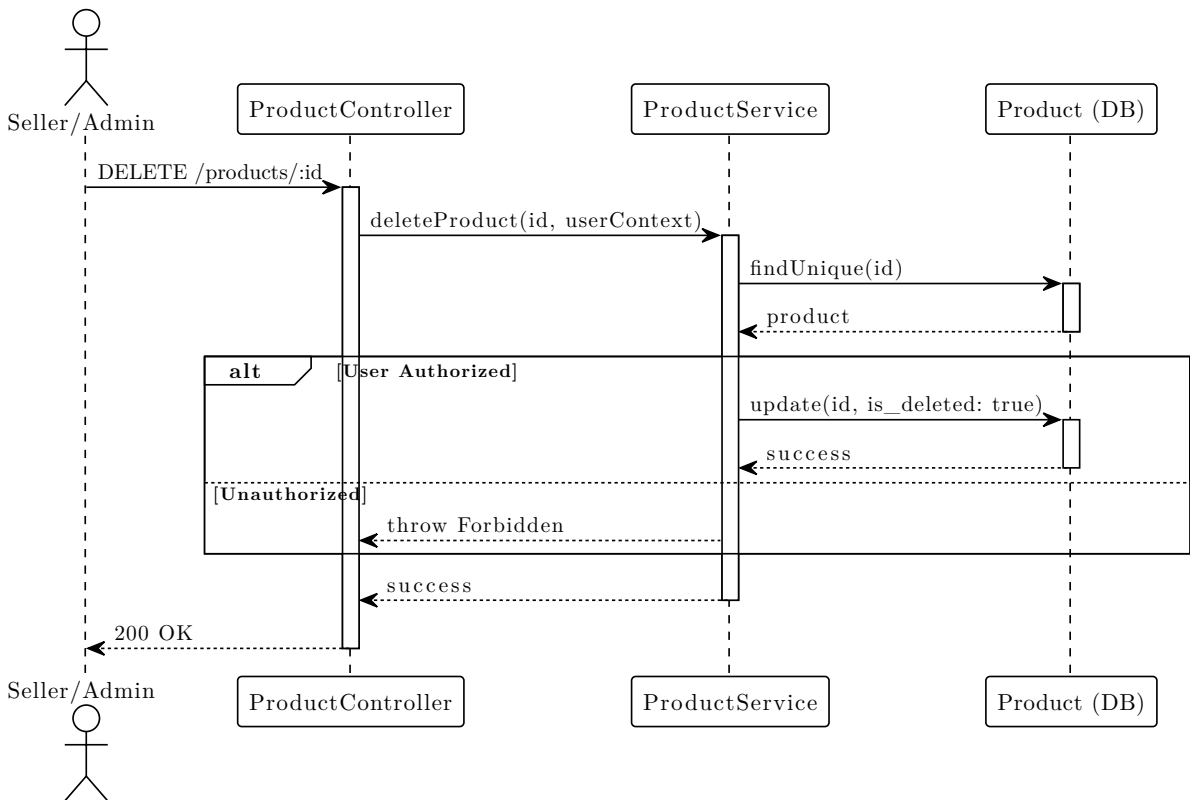


Figure 21: Delete Product Sequence

Product: Get Seller Products Sequence

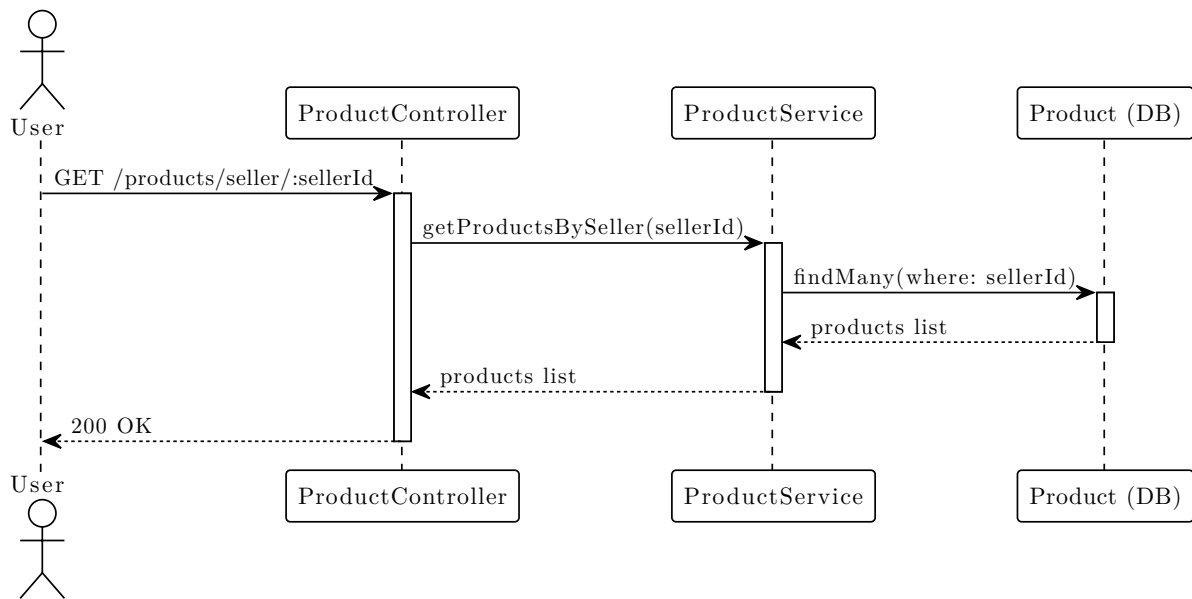


Figure 22: Get Seller Products Sequence

4.2.1.3. Order Management Module

- **Responsibilities:** Lifecycle from cart to delivery.
- **Key Features:** Cart management, Order placement, Status tracking.

Order: Add to Cart Sequence

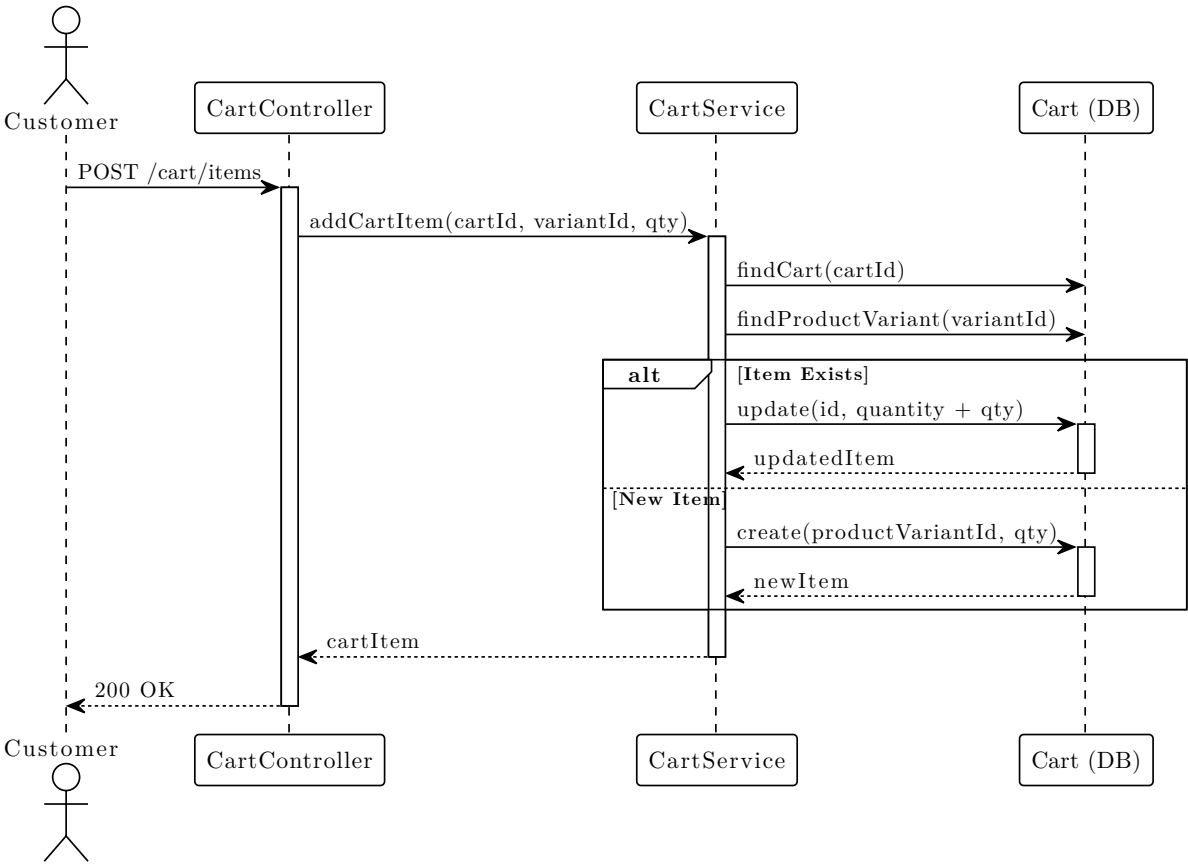


Figure 23: Add to Cart Sequence

Order: View Cart Sequence

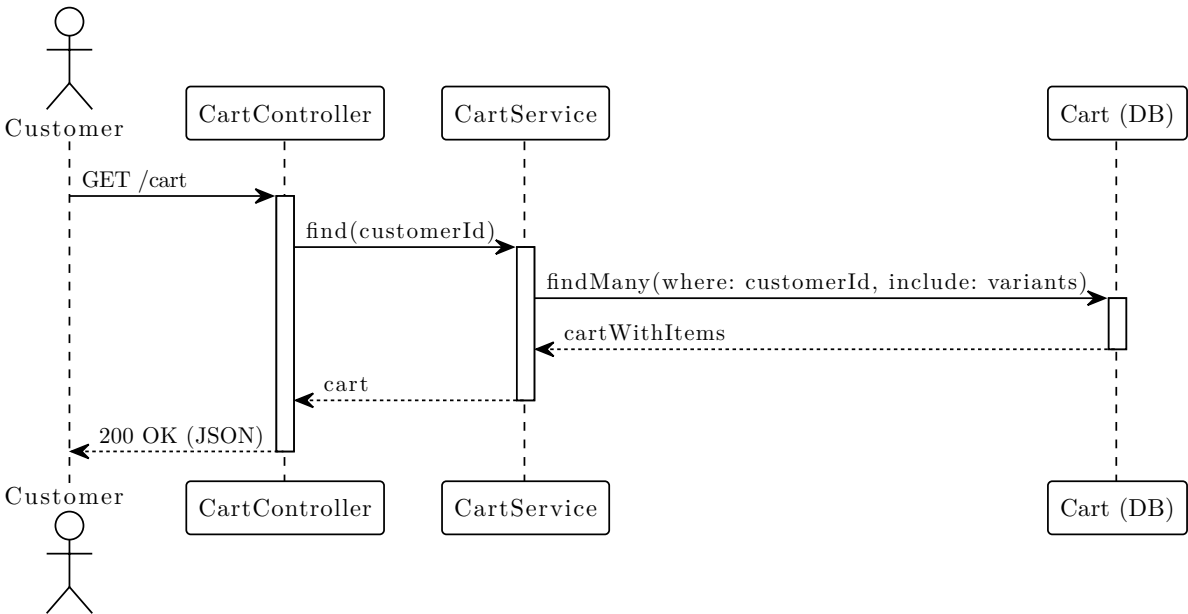


Figure 24: View Cart Sequence

Order: Update Cart Item Sequence

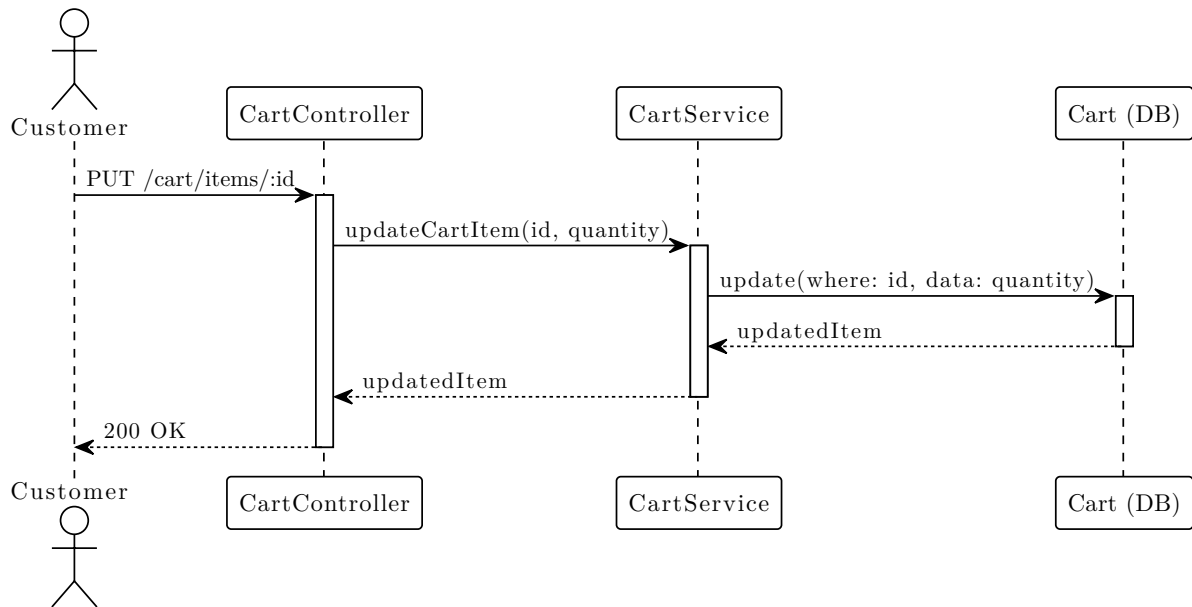


Figure 25: Update Cart Item Sequence

Order: Remove from Cart Sequence

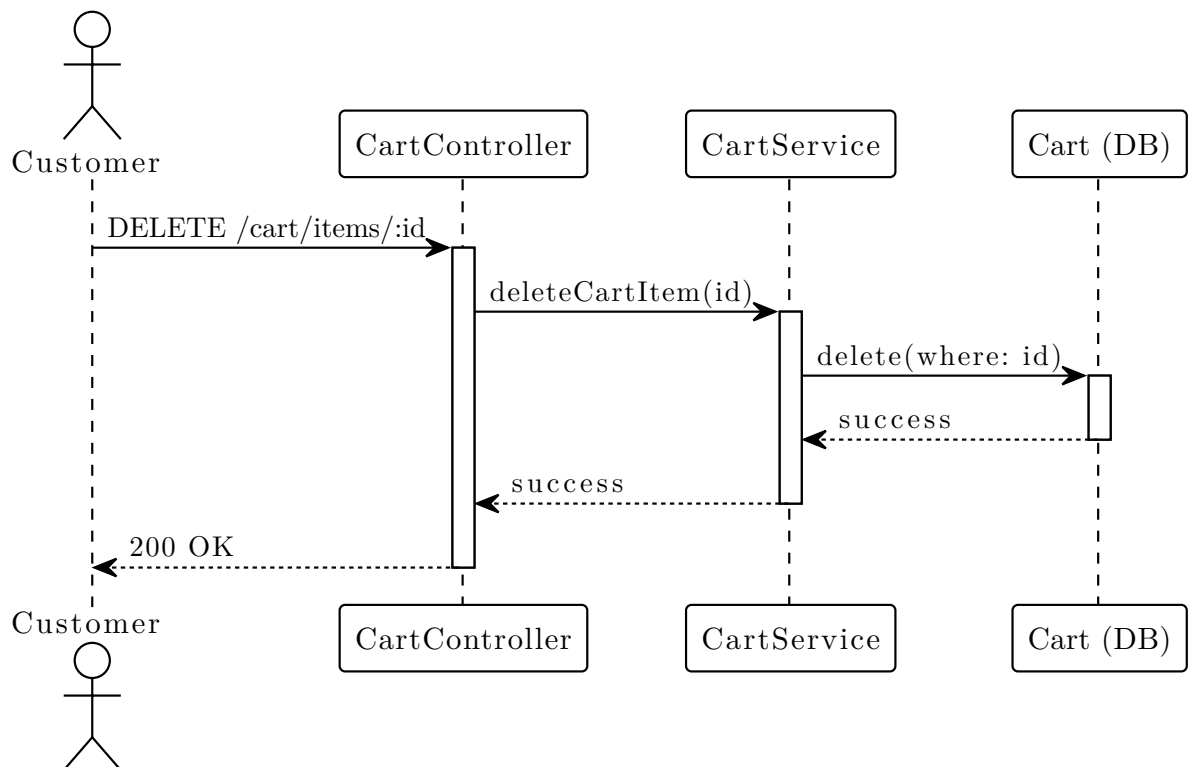


Figure 26: Remove from Cart Sequence

Order: Checkout Sequence

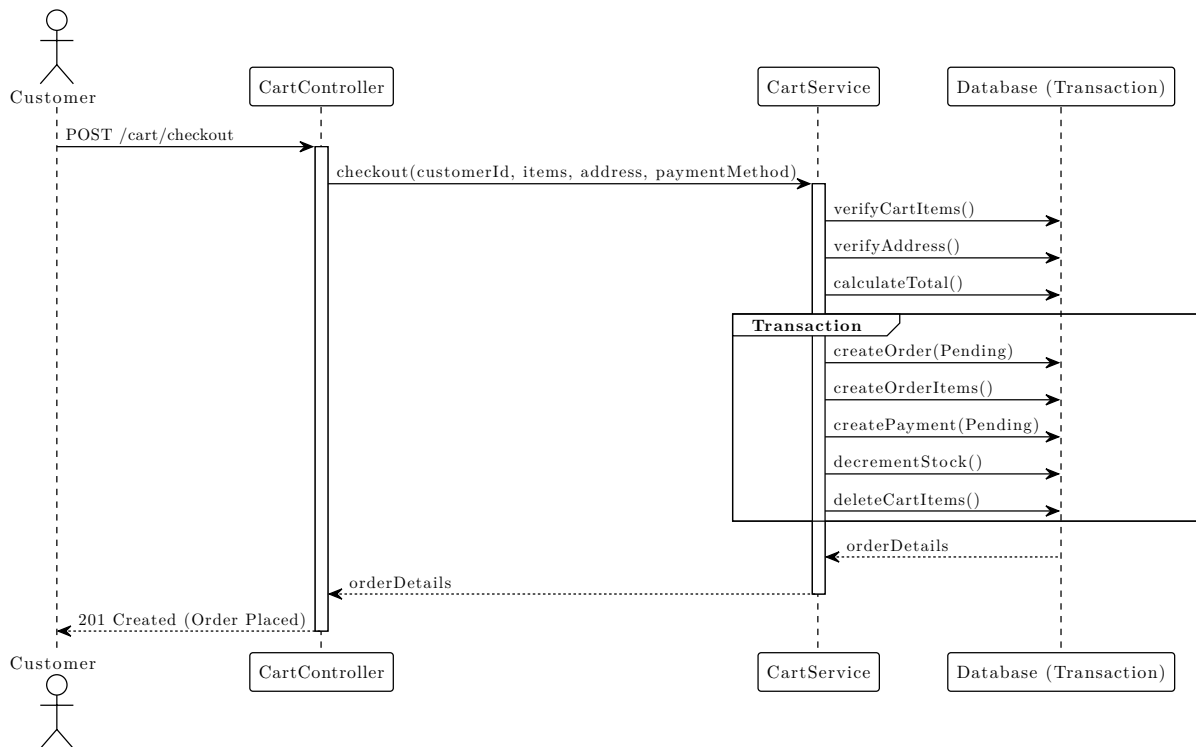


Figure 27: Order Checkout Sequence

Order: History Sequence

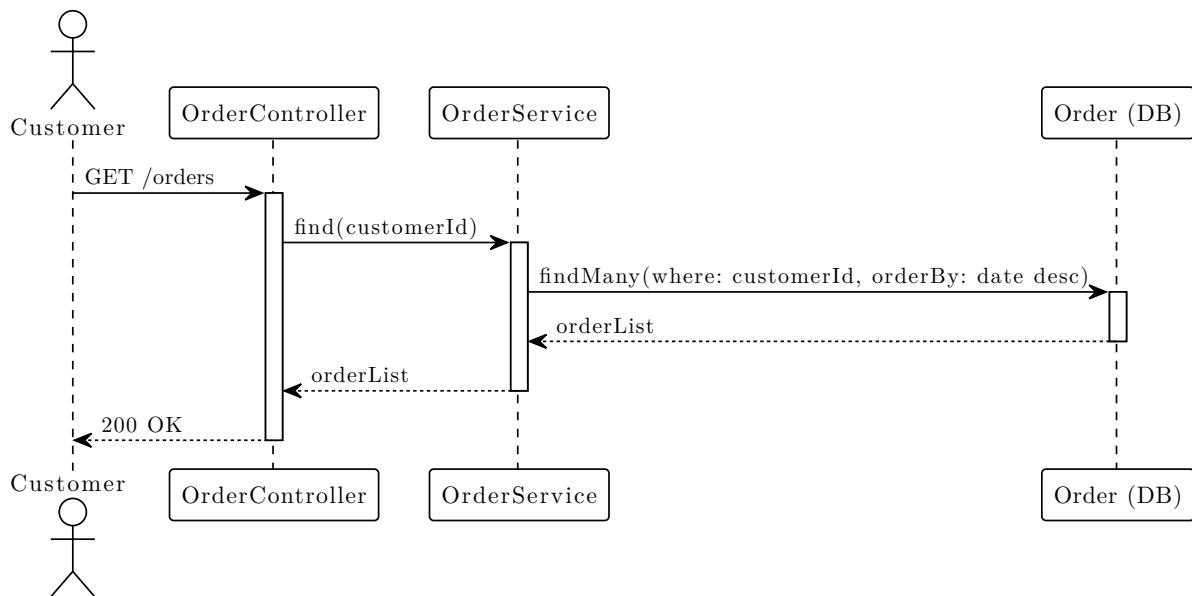


Figure 28: Order History Sequence

Order: Detail Sequence

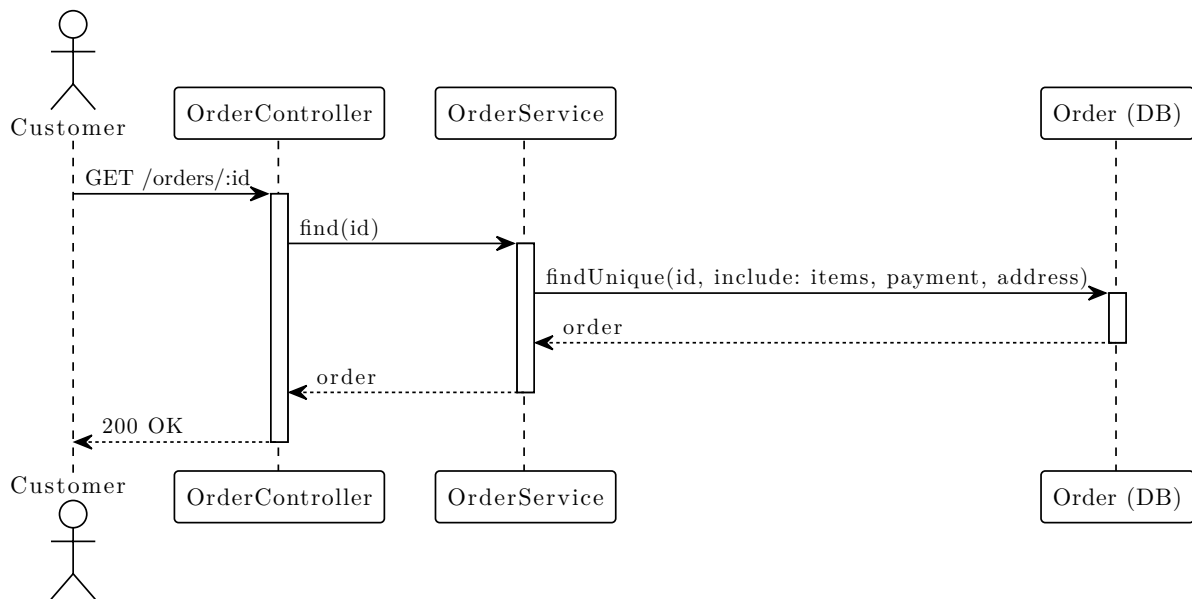


Figure 29: Order Detail Sequence

Order: Update Status Sequence

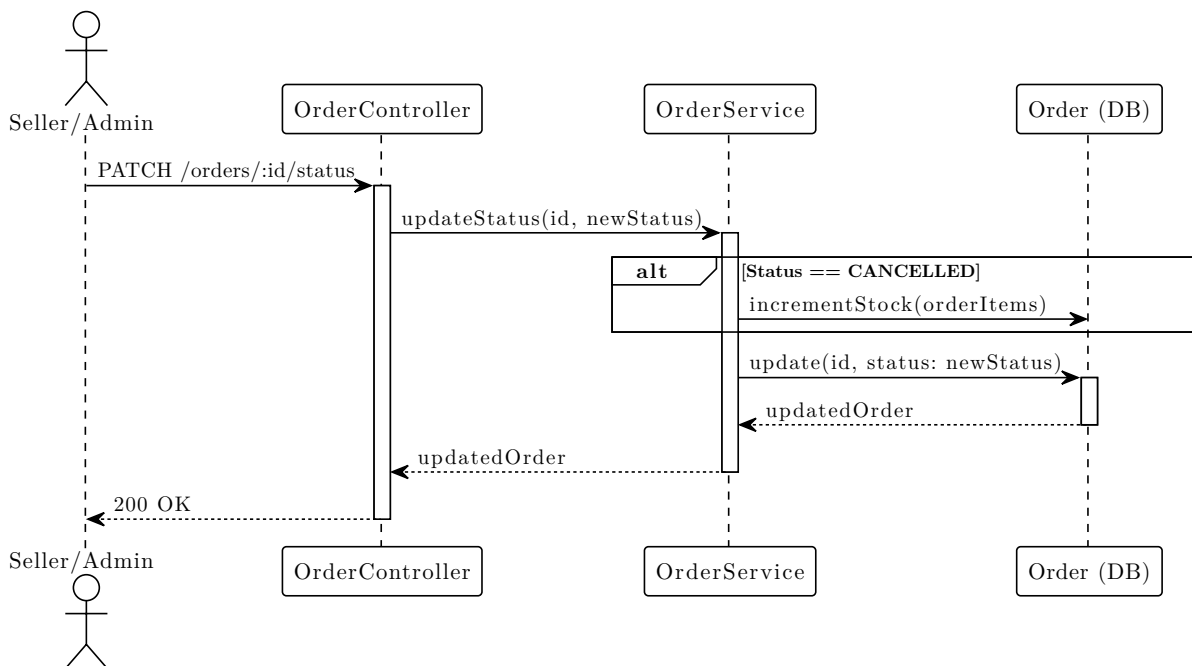


Figure 30: Update Order Status Sequence

Order: Cancel Sequence

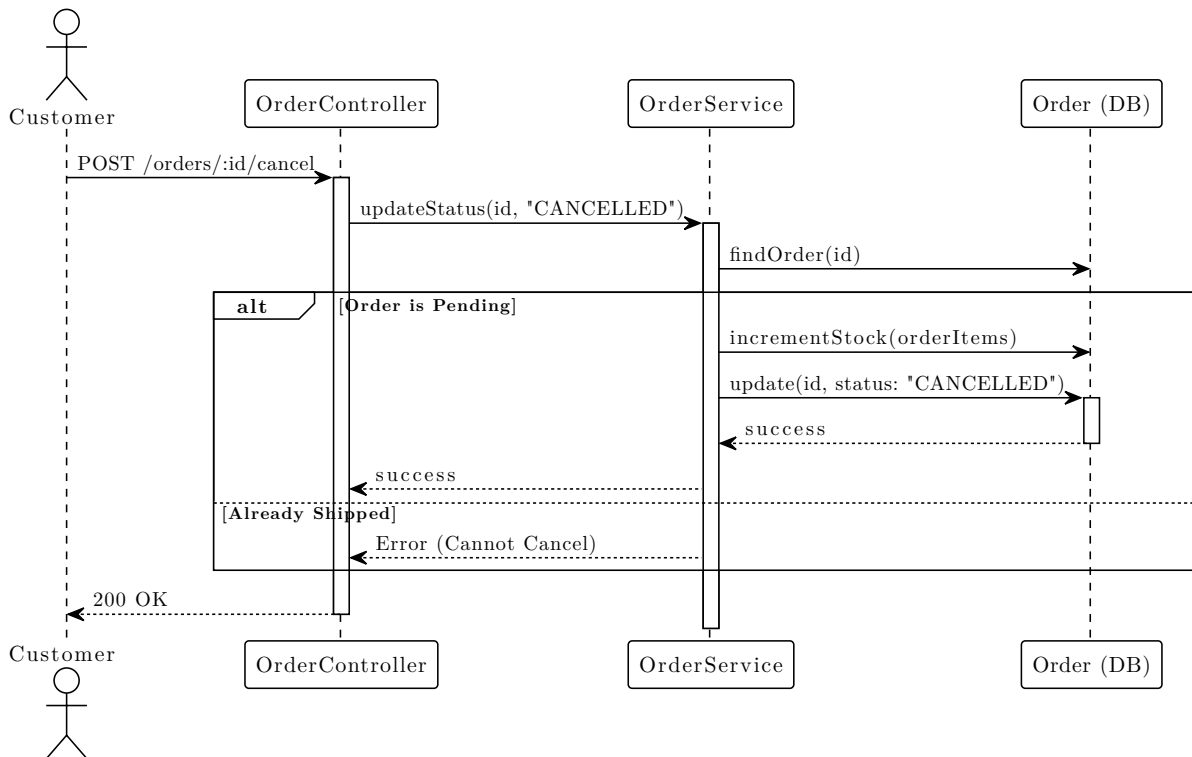


Figure 31: Cancel Order Sequence

4.2.1.4. Payment Module

- **Responsibilities:** Handling transactions and integrations.
- **Key Features:** Multiple methods (COD, VNPay), Secure callbacks.

Payment: Initiate Sequence

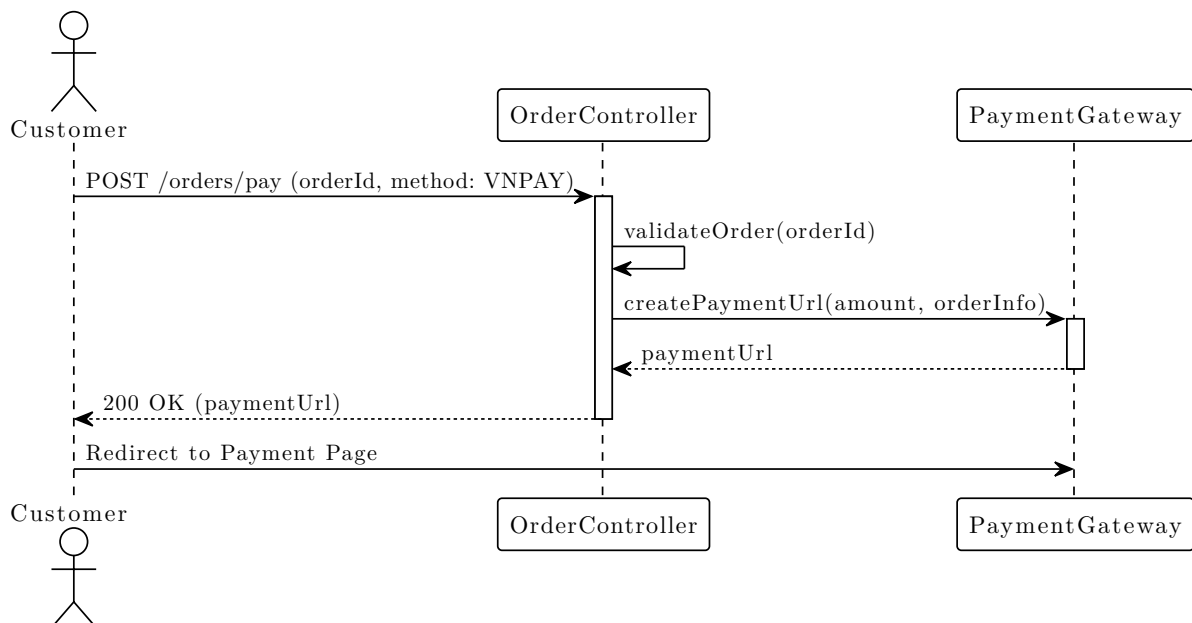


Figure 32: Payment Initiation Sequence

4.2.1.5. Promotion Module

- **Responsibilities:** Discount and coupon management.
- **Key Features:** Flash sales, Coupon codes.

Promotion: Create Sequence

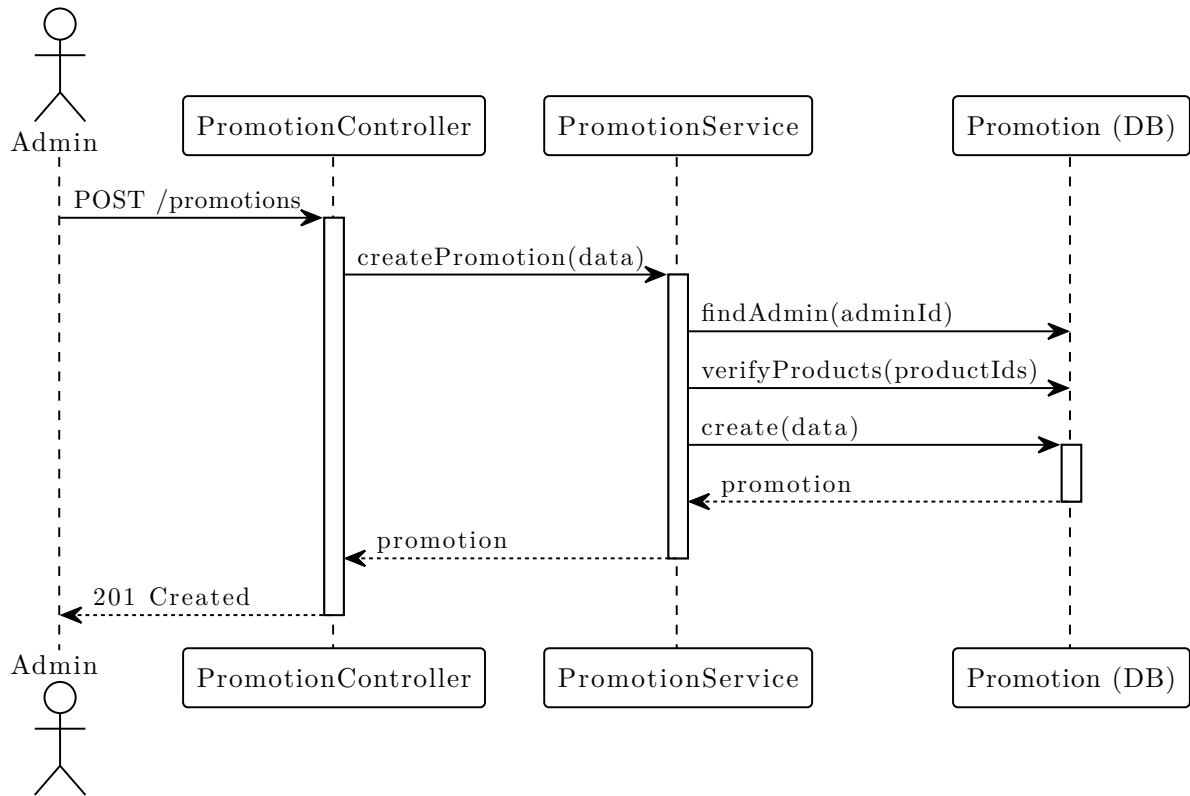


Figure 33: Create Promotion Sequence

Promotion: Create Coupon Sequence

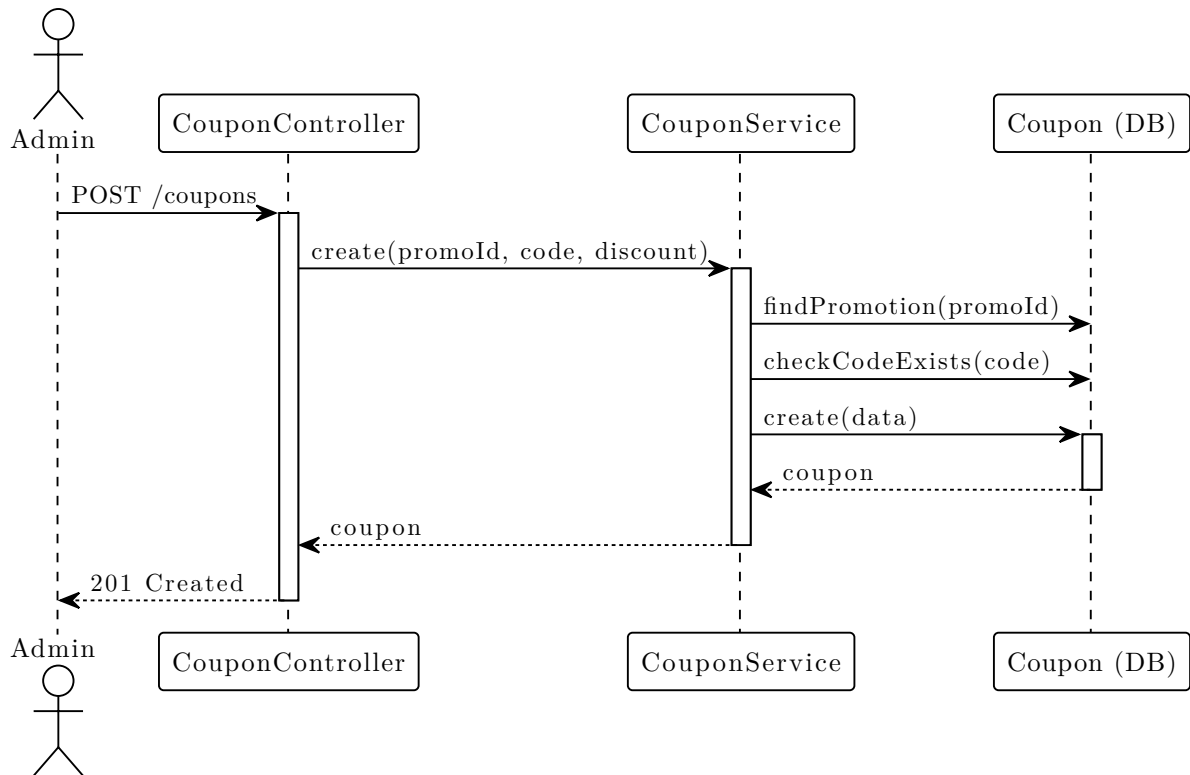


Figure 34: Create Coupon Sequence

Promotion: List Sequence

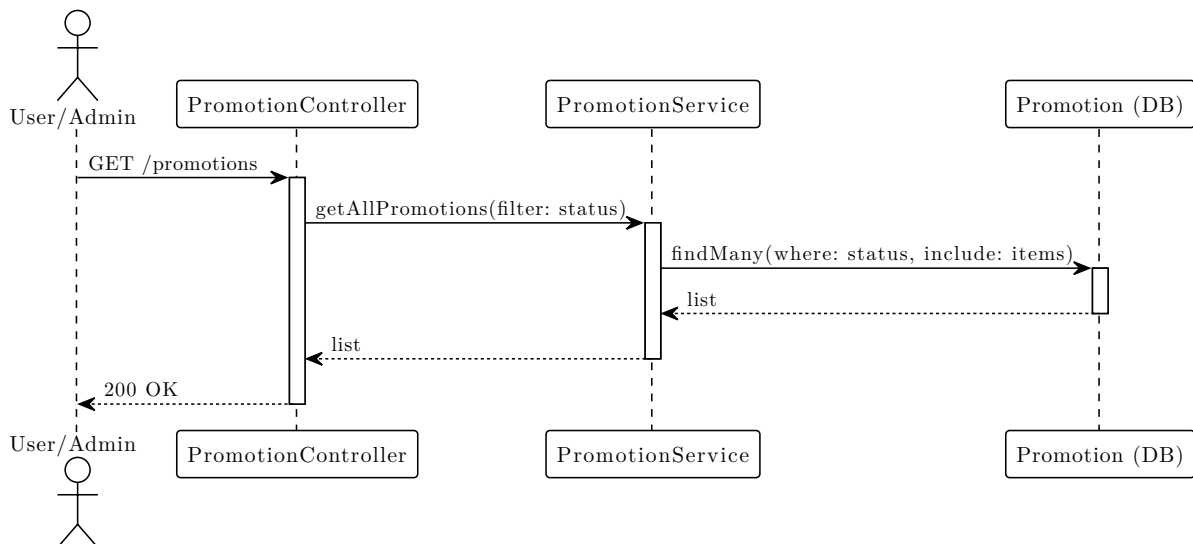


Figure 35: List Promotions Sequence

4.2.1.6. Reporting & Analytics Module

- **Responsibilities:** Insights for admins and sellers.
- **Key Features:** Sales charts, User growth, Top products.

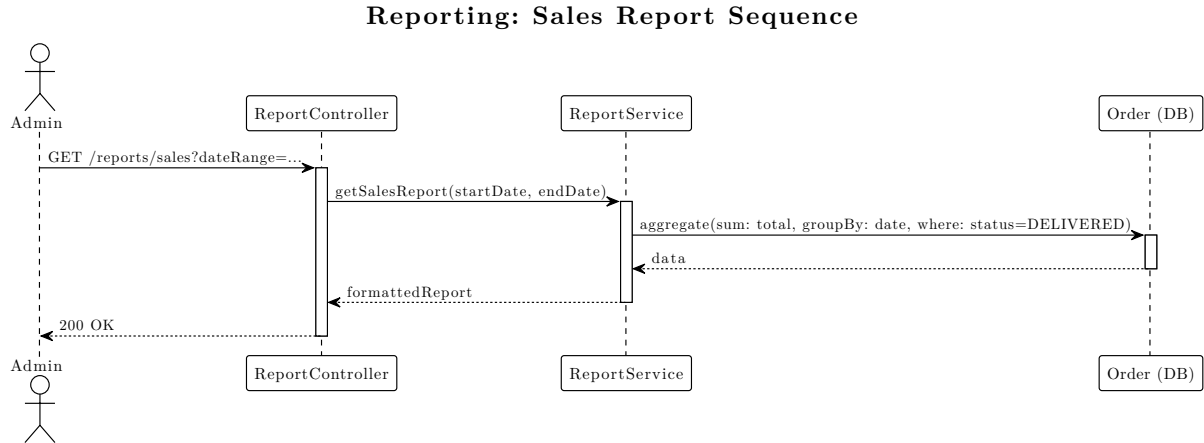


Figure 36: Sales Report Sequence

4.3. Data Design

4.3.1. Data Design

The database schema is designed using PostgreSQL and managed via Prisma ORM. Below are the key entities and their relationships.

4.3.1.1. Key Entities

- **User:** The central entity for authentication.
 - Fields: `id`, `email`, `password`, `role` (Admin/Seller/Customer).
 - Relationships: 1-to-1 with `Customer`, `Seller`, `Admin` profiles.
- **Product:** Represents items for sale.
 - Fields: `id`, `name`, `price`, `stockQuantity`, `images`.
 - Relationships: Belongs to `Category`, created by `Seller`.
- **Order:** Represents a purchase transaction.
 - Fields: `id`, `totalAmount`, `status`, `orderDate`.
 - Relationships: Belongs to `Customer`, contains multiple `OrderItems`.
- **OrderItem:** A specific product variant within an order.
 - Fields: `quantity`, `price` at time of purchase.
 - Relationships: Links `Order` and `ProductVariant`.
- **Payment:** Tracks the financial transaction for an order.
 - Fields: `amount`, `method`, `status`.
 - Relationships: 1-to-1 with `Order`.

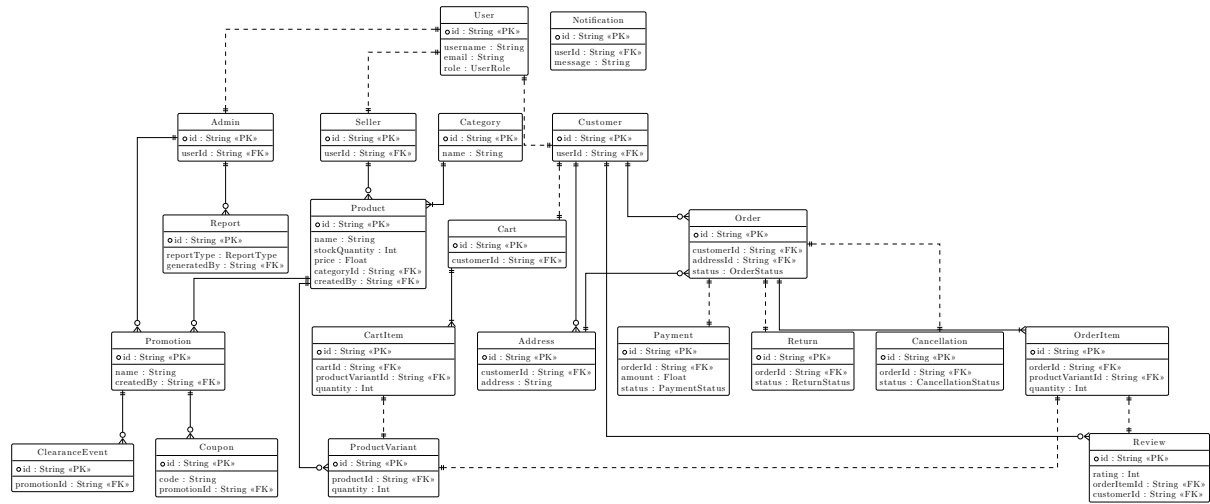


Figure 37: Entity Relationship Diagram

5. IMPLEMENTATION AND DEPLOYMENT

5.1. Development Environment

To ensure a smooth implementation process and consistent behavior across different machines, a standardized development environment was established. This section outlines the hardware and software requirements, along with the specific tools and technologies utilized.

5.2. Software Configuration

The following software tools and runtimes are required to build and run the system:

Software	Version	Purpose
Docker	v24.x	Containerization platform for consistent environments
Docker Compose	v2.x	Orchestration of multi-container applications

Table 20: Software Requirements

5.3. Development Tools

To maximize productivity and code quality, the following development tools were employed:

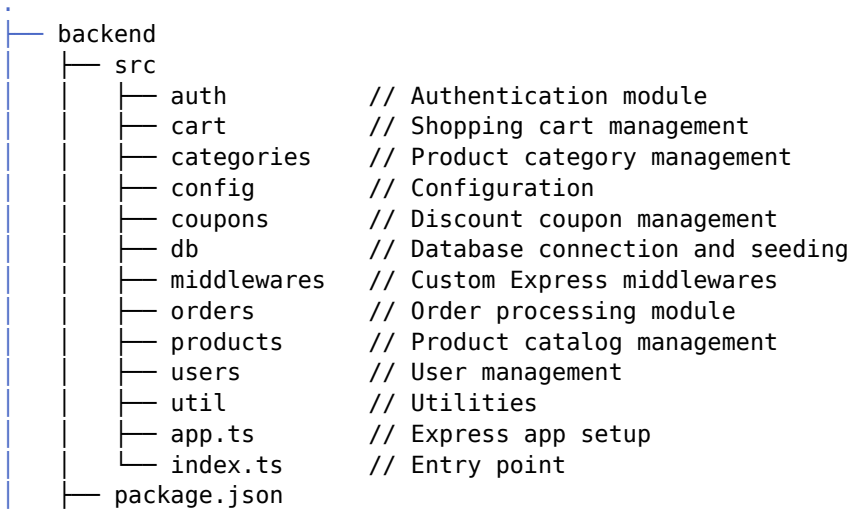
- **Visual Studio Code (VS Code):** The primary Integrated Development Environment (IDE), enhanced with extensions for ESLint, Prettier, and Docker.
- **Git:** Version control system for tracking changes and collaboration.
- **Swagger:** API testing tool used to verify backend endpoints before frontend integration.
- **Prisma Studio:** Database management tools for inspecting schemass.

5.4. Source Code Structure

The source code for “The Everything Shop” is organized as a monorepo, containing both the backend API and the frontend client. This structure facilitates easier development and deployment. The root directory contains the `docker-compose.yml` files for orchestration, while the application logic is split into `backend/` and `frontend/` directories.

5.5. Project Tree

The following tree structure represents the actual organization of the source code:



```

├── tsconfig.json
├── frontend
│   ├── src
│   │   ├── assets          // Static assets
│   │   ├── components      // Reusable UI components
│   │   ├── pages           // Page views
│   │   ├── redux           // State management
│   │   ├── services        // API service calls
│   │   ├── utils           // Helper functions
│   │   ├── App.jsx         // Main React component
│   │   └── main.jsx        // React entry point
│   ├── package.json
│   └── vite.config.js
├── docker-compose.dev.yml    // Development orchestration
├── docker-compose.prod.yml  // Production orchestration
└── traefik                  // Reverse proxy configuration

```

5.6. Backend Architecture

The backend adopts a **Feature-Based Architecture**. Instead of grouping files by type (Controllers, Routes, Services), they are grouped by functional module (e.g., `auth`, `products`, `orders`). This ensures that all logic related to a specific feature is co-located, improving maintainability and scalability.

- **Module Structure:** Each module folder (e.g., `src/products`) typically contains:
 - `*.controller.ts`: Handles HTTP requests and responses.
 - `*.service.ts`: Contains business logic and database interactions.
 - `*.route.ts`: Defines API endpoints.
 - `*.schema.ts`: Validation schemas.

5.7. Frontend Architecture

The frontend follows a Component-Based Architecture standard in React applications.

- **components/**: Contains atomic and molecule components reused across pages.
- **pages/**: Separated by user role (`admin`, `customer`, `seller`) to manage different views effectively.
- **services/**: Abstracts API calls, keeping components clean of networking logic.

5.8. Implementation of Main Functions

This section highlights the code implementation of the system's most critical functions, demonstrating how the actual core business logic is handled in the codebase.

5.8.1. Authentication Module

The authentication system secures the application by verifying user identities, issuing JWT tokens (access and refresh), and managing cookies.

5.8.1.1. Login Controller

The `login` function validates user credentials, generates tokens, sets secure cookies, and retrieves complete user profile information (including customer, seller, or admin roles).

```

// backend/src/auth/auth.controller.ts
static async login(req: Request, res: Response) {
  const { email, password } = req.body as z.infer<typeof loginSchema>;

  try {
    const { user, accessToken, refreshToken } = await AuthService.login(email,
password);

```



```

    setAuthCookies(res, accessToken, refreshToken);

    // Fetch complete user info including customer/seller/admin relations
    const completeUser = await prisma.user.findUnique({
      where: { id: user.id },
      select: {
        id: true,
        username: true,
        email: true,
        role: true,
        createdAt: true,
        updatedAt: true,
        customer: { select: { id: true, image: true, addresses: true } },
        seller: { select: { id: true, email: true, image: true } },
        admin: { select: { id: true } }
      },
    });

    return Send.success(res, completeUser);
  } catch (error: any) {
    if (error.message === "Invalid email or password.") {
      return Send.unauthorized(res, null, error.message);
    }
    logger.error({ error }, "Login Failed");
    return Send.error(res, null, error.message || "Login failed.");
  }
}

```

5.8.2. Product Management

This module handles the retrieval and management of the product catalog.

5.8.2.1. Get All Products

The `getAllProducts` function supports pagination, category filtering, and search functionality, validating input query parameters before delegating to the service layer.

```

// backend/src/products/product.controller.ts
static async getAllProducts(req: Request, res: Response) {
  try {
    const queryValidation = ProductSchema.getAllProductsQuery.safeParse(req.query);

    if (!queryValidation.success) {
      const errors = queryValidation.error.flatten().fieldErrors;
      return Send.validationErrors(res, errors);
    }

    const { skip, take, categoryId, search } = queryValidation.data as
    GetAllProductsQuery;

    const result = await ProductService.getAllProducts(skip, take, categoryId,
    search);

    return Send.success(res, result, "Products fetched successfully");
  } catch (error) {
    logger.error({ error }, "Failed to fetch products");
    return Send.error(res, null, "Failed to fetch products");
  }
}

```

5.8.3. Order Module

The order module manages the checkout process and direct order creation.

5.8.3.1. Create Direct Order

The `createDirect` function handles immediate purchases of a single product variant. It identifies the customer from the authenticated session and delegates the order creation logic.

```
// backend/src/orders/order.controller.ts
static async createDirect(req: Request, res: Response) {
  const { addressId, productVariantId, quantity } = req.body;
  const customerId = (req as any).user?.customer?.id;

  if (!customerId) {
    return Send.notFound(res, null, "Customer not found");
  }

  try {
    const order = await OrderService.createDirectOrder(customerId, addressId,
productVariantId, quantity);

    return Send.success(res, order, "Order created successfully");
  } catch (error: any) {
    return Send.badRequest(res, null, error.message);
  }
}
```

5.8.4. Cart Management

The cart module allows customers to manage items before checkout.

5.8.4.1. Checkout Logic

The `checkout` function initiates the order process from a cart, handling multiple items, address selection, and payment method.

```
// backend/src/cart/cart.controller.ts
static async checkout(req: Request, res: Response) {
  try {
    const { customerId } = req.params;
    const { cartItemIds, addressId, paymentMethod } = req.body;

    const order = await CartService.checkout({
      customerId,
      cartItemIds,
      addressId,
      paymentMethod,
    });

    return Send.success(res, { order }, "Checkout successful");
  } catch (error: any) {
    logger.error({ error }, "Error during checkout");

    if (error.message.includes("not found") || error.message.includes("do not
belong")) {
      return Send.badRequest(res, {}, error.message);
    }

    return Send.error(res, {}, "Internal server error");
  }
}
```

5.8.5. Security Middleware

Middleware functions ensure that only authorized users can access specific resources.

5.8.5.1. Auth Guard

The `authGuard` middleware extracts the JWT token from cookies, verifies it, and attaches the user object (including role-specific profiles) to the request context.

```
// backend/src/middlewares/authGuard.ts
export const authGuard = async (req: Request, res: Response, next: NextFunction) => {
  const token = req.cookies.accessToken;
  if (!token) {
    return Send.unauthorized(res, null); // can't find token, unauthorized user
  }
  try {
    const decodedToken = jwt.verify(token, authConfig.secret) as { userId: string };

    // Fetch complete user with relations
    const user = await prisma.user.findUnique({
      where: { id: decodedToken.userId },
      include: {
        customer: true,
        seller: true,
        admin: true,
      }
    });

    if (!user) {
      return Send.unauthorized(res, { message: "User not found" });
    }

    (req as any).user = user;

    next();
  } catch (error) {
    // token is invalid or expired
    logger.error({ error }, "Authentication failed");
    return Send.unauthorized(res, null);
  }
};
```

5.9. System Deployment

The deployment of “The Everything Shop” is designed for portability, scalability, and security using containerization. The system orchestrates multiple services ensuring a seamless production environment.

5.9.1. Deployment Architecture

The system utilizes **Docker Compose** to manage a collection of interacting containers. A **Traefik Reverse Proxy** sits at the entry point of the cluster, handling routing, SSL termination (via Let’s Encrypt), and load balancing.

5.9.1.1. Core Services

The production `docker-compose.prod.yml` defines the following critical services. In a production environment, services **pull pre-built versioned images** from a container registry rather than building from source at runtime.

- **Traefik:** Acts as the Edge Router, automatically discovering services via Docker labels. It manages HTTPS certificates and routes traffic to the appropriate container based on hostnames.
- **App (Backend):** Runs the Node.js/Express API using a pre-built image. It runs database migrations (`npx prisma migrate deploy`) upon startup.
- **Frontend:** Serves the React SPA using **Nginx**. The image contains the pre-compiled static assets (`/dist`), ensuring fast startup and consistent deployments.
- **DB (PostgreSQL):** The primary relational database, persisting data to a docker volume. It includes a healthcheck to ensure dependent services wait until it is ready.
- **S3 (MinIO):** An S3-compatible object storage service for handling product images and file uploads.
- **Mailhog:** A simulated SMTP server for testing email notifications without sending actual emails.

5.9.2. Docker Configuration

5.9.2.1. Production Compose Stack

The production stack relies on pulling optimized images. This separation of concerns ensures that the deployment environment is not cluttered with build tools.

```
# docker-compose.prod.yml
services:
  app:
    image: {registry}/tes-backend:latest
    restart: unless-stopped
    env_file: .env
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.backend.rule=Host(`${DOMAIN}`) && PathPrefix(`/api`)"
      - "traefik.http.routers.backend.entrypoints=websecure"

  frontend:
    image: {registry}/tes-frontend:latest
    restart: unless-stopped
    labels:
      - "traefik.http.routers.frontend.rule=Host(`${DOMAIN}`)"
```

5.9.2.2. Immutable Artifacts (Dockerfile)

Although images are pulled in production, they are built from immutable definitions using multi-stage Dockerfiles.

```
# backend/prod.Dockerfile (Snippet)
FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/dist ./dist
# ... copy prisma and package.json ...
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/index.js"]
```

5.9.3. Environment Configuration

Sensitive configuration is decoupled from the code using `.env` files.

- **POSTGRES_USER / POSTGRES_PASSWORD:** Database credentials.
- **ACME_EMAIL:** Email used for Let's Encrypt certificate registration.
- **DOMAIN_NAME:** The base domain for routing.

5.9.4. Deployment Diagram (Textual Representation)

The architecture follows this flow:

1. **Client Request** (Browser) -> **Internet**
2. **Traefik** (Port 443) -> **Route Matching**
 - Host localhost -> **Frontend** (Nginx)
 - Path /api -> **Backend** (Node.js)
 - Host obj.storage... -> **MinIO** (S3)
 - Host mail... -> **MailHog**
3. **Backend** -> **DB** (PostgreSQL 5432)
4. **Backend** -> **S3** (MinIO 9000)

6. TESTING AND EVALUATION

6.1. Testing Strategy

To ensure the reliability and correctness of “The Everything Shop,” a multi-layered testing strategy was employed. This approach combines functional verification with structural analysis to identify defects at different granularities.

6.1.1. Testing Levels

6.1.1.1. Unit Testing

Unit tests focus on individual components and functions in isolation.

- **Scope:** Backend utility functions, data validation helpers, and Redux reducers.
- **Method:** White-box testing using Jest (implied).
- **Goal:** Verify that specific logic blocks (e.g., price calculation, email validation) return correct outputs for given inputs.

6.1.1.2. Integration Testing

Integration tests verify the interactions between different modules/services.

- **Scope:** API endpoints, Database interactions, and Client-Server communication.
- **Method:** Black-box testing via Postman and internal API tests.
- **Goal:** Ensure that the Express backend correctly communicates with the PostgreSQL database and that the React frontend successfully consumes these APIs.

6.1.1.3. System Testing

System tests evaluate the complete, integrated application against the functional requirements.

- **Scope:** Full user flows (e.g., “Registration to Checkout”).
- **Method:** Black-box testing via the web interface.
- **Goal:** Validate that the system meets user needs as defined in the Use Case Specifications.

6.1.2. Testing Environment

- **OS:** Linux (Development)
- **Browser:** Chromium / Firefox
- **Database:** PostgreSQL 16 (Dockerized)
- **Tools:** Jest (Unit), Manual Execution (System)

6.2. Test Case Construction and Execution

This section details the specific test cases designed to validate the system’s functionality.

6.2.1. Authentication Module Tests

- **TC-01: Verify User Registration with valid data**
 - **Input Data:** Name: “John”, Email: “test@mail.com”, Pass: “123456”
 - **Expected Result:** Account created, redirect to Login
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-02: Verify Registration with duplicate email**
 - **Input Data:** Email: “test@example.com” (existing)
 - **Expected Result:** Error: “Email already exists”
 - **Actual Result:** Error: “User exists”
 - **Status:** Pass

- **TC-03: Verify Login with valid credentials**
 - **Input Data:** Email: “test@example.com”, Pass: “123456”
 - **Expected Result:** JWT Token received, Redirected to Home page
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-04: Verify Login with invalid password**
 - **Input Data:** Email: “test@example.com”, Pass: “wrong”
 - **Expected Result:** Error: “Invalid credentials”
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-05: Verify Logout**
 - **Input Data:** Click “Logout” button
 - **Expected Result:** Session cleared, redirect to Home
 - **Actual Result:** As Expected
 - **Status:** Pass

6.2.2. Product Module Tests

- **TC-06: Admin creates product with valid data**
 - **Input Data:** Name: “Laptop”, Price: 999, Stock: 10
 - **Expected Result:** Product added to catalog
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-07: Verify Negative Price input**
 - **Input Data:** Price: −100
 - **Expected Result:** Validation Error: “Price must be positive”
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-08: Delete Product**
 - **Input Data:** Select Product -> Delete
 - **Expected Result:** Product removed from list
 - **Actual Result:** As Expected
 - **Status:** Pass

6.2.3. Ordering & Checkout Tests

- **TC-09: Add Item to Cart**
 - **Input Data:** Select “Laptop”, Qty: 1
 - **Expected Result:** Cart Icon count +1
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-10: Add quantity exceeding stock**
 - **Input Data:** Stock: 10, Request: 11
 - **Expected Result:** Error: “Insufficient stock”
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-11: Calculate Cart Total**
 - **Input Data:** Item A (\$10) + Item B (\$20)
 - **Expected Result:** Total: \$30

- **Actual Result:** Total:\$30
- **Status:** Pass
- **TC-12: Complete Checkout**
 - **Input Data:** Confirm Payment -> Submit
 - **Expected Result:** Order 123 created, Cart cleared
 - **Actual Result:** As Expected
 - **Status:** Pass
- **TC-13: Verify Stock Deduction**
 - **Input Data:** Checkout 1 “Laptop”
 - **Expected Result:** Stock reduces by 1
 - **Actual Result:** As Expected
 - **Status:** Pass

6.2.4. Security & Permissions Tests

- **TC-14: Customer accessing Admin Page**
 - **Input Data:** Navigate to /admin/dashboard
 - **Expected Result:** Redirect to Home / 403 Forbidden
 - **Actual Result:** Redirected
 - **Status:** Pass
- **TC-15: SQL Injection on Login**
 - **Input Data:** Email: ' OR '1'='1
 - **Expected Result:** Login Failed (Sanitized)
 - **Actual Result:** Safe
 - **Status:** Pass
- **TC-16: Access Protected API without Token**
 - **Input Data:** GET /api/orders
 - **Expected Result:** 401 Unauthorized
 - **Actual Result:** As Expected
 - **Status:** Pass

7. CONCLUSION AND FUTURE DEVELOPMENT

7.1. Conclusion

The “Everything Shop” project has been a comprehensive exercise in full-stack web application development. Throughout this course, we have successfully navigated the entire software development lifecycle, from requirements elicitation and system design to implementation and verification.

Key Achievements:

1. **Architecture Mastery:** We successfully implemented a Monolithic MVC architecture, establishing a clean separation of concerns between our Postgres data layer, Express logic layer, and React presentation layer.
2. **Technology Proficiency:** The project demonstrates solid competence in the PERN stack, leveraging TypeScript for type safety and Prisma for efficient database interactions.
3. **Educational Value:** Beyond the code, this project provided invaluable insights into state management, authenticating users securely with JWT, and handling complex object relationships (e.g., Sellers owning Products which are in Customers’ Orders).

By delivering a functioning, containerized application, we have met the primary objectives of the course and laid a strong foundation for future software engineering endeavors.

7.2. Limitations of the Project

Despite the successful deployment, the system operates under significant constraints inherent to its academic scope and on-premise infrastructure.

Functional Constraints:

- **Simplistic Logic:** The product categorization system is rigid, supporting only basic hierarchy without dynamic attributes or tagging. Similarly, shop functions like inventory management lack depth (e.g., no batch updates or supplier management).
- **Scope Limits:** Essential e-commerce features such as real-time shipping calculation, tax automation, and third-party payment gateways are simulated or omitted.

Technical Limitations:

- **Performance:** The frontend application suffers from noticeable latency. This sluggishness is attributed to unoptimized React re-renders and the overhead of the current development environment.
- **Error Handling:** The system’s resilience is low. Error messages are often generic (e.g., “500 Server Error” or “Login Failed”), providing little actionable feedback to the user or developer for debugging.
- **Infrastructure:** Hosted on non-redundant on-premise servers, the application lacks the high availability and scalability guarantees of a cloud-native deployment.

7.3. Future Development

To evolve “The Everything Shop” from an academic project to a production-grade platform, several strategic improvements are recommended:

1. Architectural Evolution (Microservices):

- **Strategy:** Decompose the monolithic backend into dedicated services (e.g., `AuthService`, `OrderService`, `InventoryService`).
- **Benefit:** This would improve scalability, isolate failures, and allow independent deployment of critical components.

2. **AI & Personalization:**

- **Strategy:** Integrate a Recommendation Engine to analyze viewing history and suggest relevant products.
- **Benefit:** Enhances user engagement and increases conversion potential.

3. **Payment Integration:**

- **Strategy:** Replace the mock payment logic with secure Stripe or PayPal API integration.
- **Benefit:** Enables actual revenue generation and supports diverse payment methods.

4. **UX Optimization:**

- **Strategy:** Refactor the frontend with performance profiling, implementing lazy loading and better state management (e.g., React Query) to address current latency issues.
- **Benefit:** significantly improved user retention and satisfaction.

8. REFERENCES

Technologies & Tools:

1. **React Documentation.** (2025). “React: The Library for Web and Native User Interfaces.” Available at: <https://react.dev/>
2. **Express.js.** (2025). “Fast, unopinionated, minimalist web framework for Node.js.” Available at: <https://expressjs.com/>
3. **Prisma Data Guide.** (2025). “ORM for Node.js & TypeScript.” Available at: <https://www.prisma.io/>
4. **PostgreSQL.** (2025). “The World’s Most Advanced Open Source Relational Database.” Available at: <https://www.postgresql.org/>
5. **Docker Documentation.** (2025). “Containerize your applications.” Available at: <https://docs.docker.com/>